

PROVA DE APTIDÃO PROFISSIONAL

GoDEEPER



CURSO PROFISSIONAL DE TÉCNICO DE GESTÃO E PROGRAMAÇÃO DE SISTEMAS INFORMÁTICOS

NOME ALUNO: Gabriel Marinho Bispo

N.º: 11

PROFESSOR ORIENTADOR: Luís Fernandes

ANO LETIVO: 2025/2026



DEDICATÓRIA

A Deus, de quem vem toda a sabedoria, toda a inspiração e toda a força. Este projeto é, antes de qualquer outra coisa, uma oferta a Ti.

Ao Professor Ângelo Mateus e ao Professor Luís Fernandes, pelo acompanhamento próximo, pela paciência, pelos conselhos e por nunca terem deixado de acreditar no meu trabalho. A vossa orientação foi essencial para que este projeto chegasse onde chegou, e a vossa preocupação genuína com o meu crescimento fez toda a diferença neste percurso.

À minha irmã Isabel, por teres dado alma, identidade e beleza a este projeto - desde as cores que o definem, aos logótipos que o representam, aos produtos do site. Foste a minha guia criativa, o meu apoio constante. Obrigado por seres quem és.



AGRADECIMENTOS

Este projeto não teria sido possível sem o contributo, o apoio e a presença de pessoas que tornaram este percurso mais rico e significativo. A todas elas, deixo aqui o meu sincero agradecimento.

Em primeiro lugar, e acima de tudo, agradeço a Deus. Este projeto nasceu de uma ideia que Ele colocou no meu coração, e é com a consciência de que tudo o que faço deve glorificá-Lo que o apresento. Que o GoDeeper seja, acima de qualquer competência técnica demonstrada, um reflexo do meu desejo de servir e honrar o Seu nome - em cada linha de código, em cada decisão tomada e em cada funcionalidade desenvolvida. Que esta e todas as minhas ações, profissionais e pessoais, sejam sempre para a Sua glória.

À minha irmã Isabel, um agradecimento especial e do fundo do coração. Foi ela quem deu vida à identidade visual do GoDeeper - desde a paleta de cores que define o projeto, aos logótipos, aos produtos personalizados, às edições de vídeo e fotografia. A beleza que o GoDeeper transmite tem o teu toque, a tua criatividade e o teu talento. Foste muito mais do que uma colaboradora; foste a minha guia ao longo de todo este caminho.

À Liderança do grupo GoDeeper, pelo apoio constante, pelas ideias partilhadas e pela confiança depositada neste projeto. Saber que havia pessoas a acreditar nesta visão foi uma motivação que nunca deixou de me impulsionar.

Aos meus professores, um agradecimento genuíno e sentido. As vossas ideias, a vossa orientação e, acima de tudo, a vossa preocupação real com o progresso dos alunos fizeram toda a diferença. São o tipo de professores que marcam - aqueles que se importam de verdade - e esse cuidado reflete-se no resultado final deste trabalho.

À minha família, pelo apoio incondicional em todos os momentos e por nunca me deixarem esquecer a importância de concluir esta etapa. A vossa presença e encorajamento foram o alicerce que me sustentou ao longo de todo este processo.

A TODOS MUITO OBRIGADO!!!

"Dai graças ao Senhor, porque Ele é bom; o Seu amor dura para sempre." → Salmos 107:1.



TGP SI
CURSO PROFISSIONAL

REPÚBLICA
PORTUGUESA

educação



Os Fundos Europeus mais próximos de si.





ÍNDICE

Dedicatória.....	I
Agradecimentos	II
Índice	III
Índice de figuras	IV
Índice de tabelas	VII
Lista de siglas e acrónimos	VIII
1. Introdução.....	9
1.1. Fundamentação	10
1.2. Descrição do projeto	10
1.3. Objetivos do projeto	11
1.4. Estado do conhecimento	12
2. Planificação do projeto	16
2.1. Calendarização	16
2.2. Tecnologia utilizada.....	18
2.3. Protótipo	26
2.4. Base de Dados	32
3. Implementação do projeto	40
4. Problemas e soluções encontradas	67
5. Conclusão	73
Referências bibliográficas.....	76

ÍNDICE DE FIGURAS

Figura 1 - Logo GoDeeper	9
Figura 2 - Logo HTML.....	19
Figura 3 - Logo TailwindCSS	19
Figura 4 - Logo Java Script.....	20
Figura 5 - Logo Node.JS.....	20
Figura 6 - Logo Python.....	21
Figura 7 - Logo MySQL.....	21
Figura 8 - Logo Express JS.....	22
Figura 9 - Logo Multer.....	22
Figura 10 - Logo Flask	23
Figura 11 - Logo VLC	23
Figura 12 - JS Fetch API	24
Figura 13 - Logo JSON.....	24
Figura 14 - Logo Claude.ai.....	25
Figura 15 - Theme GoDeeper	26
Figura 16 - Página Inicial.....	27
Figura 17 - Página Inicial: Sobre Nós.....	27
Figura 18 - Mapa Interativo do Site: https://www.baptistasdocentro.pt/mapa	28
Figura 19 - Página de Login.....	28
Figura 20 - Página de Criação de Conta.....	29
Figura 21 - Página Principal	29
Figura 22 - Página Principal: Liderança	29
Figura 23 - Página Principal: Explorador de Mapas	30
Figura 24 - GoMusic.....	30
Figura 25 - Produtos	31
Figura 26 - Sobre a Conta	31
Figura 27 - Base de Dados p.1.....	33
Figura 28 - Base de Dados p.2	33
Figura 29 - Base de Dados p.3	34
Figura 30 - Base de Dados p.4	34
Figura 31 - Base de Dados p.5.....	35
Figura 32 - Base de Dados p.6	35
Figura 33 - Base de Dados p.7	36
Figura 34 - Base de Dados p.8	36
Figura 35 - server.js: Criação da Tabela: sessions.....	37
Figura 36 - BD Completa	39
Figura 37 - Organização das Pastas p.1	41
Figura 38 - Organização das Pastas p.2	41

Figura 39 - Organização das Pastas p.3	42
Figura 40 - Organização das Pastas p.4	42
Figura 41 - Verificação da Sessão a Cada 5 segundos	45
Figura 42 - Criação de Conta.....	46
Figura 43 - Informações do Terminal da Criação da Conta	46
Figura 44- server.js	47
Figura 45 - Terminal: Iniciar Projeto	47
Figura 46 - Login-GoDeeper.js.....	48
Figura 47 - apiCriarConta.js.....	48
Figura 48 - verificarSessao.js	49
Figura 49 - Informação do Terminal da Ação da Playlist.....	50
Figura 50 – Playlist	51
Figura 51 - Criação de Playlist.....	51
Figura 52 - Playlist Aberta.....	52
Figura 53 - Excluir Playlist.....	52
Figura 54 - Remover Música da Playlist	53
Figura 55 - Ações da Escolha das Músicas.....	53
Figura 56 - <i>playerManager.js</i>	54
Figura 57 - <i>musicas.js</i>	55
Figura 58 - <i>player.py</i>	56
Figura 59 - <i>player.py</i>	56
Figura 61 - Criação de Eventos – Administração	58
Figura 62 - Criação de Conteúdo – Administração	58
Figura 63 - Criação de Produtos - Administração.....	59
Figura 64 - Interesse: Número para o Pedido	59
Figura 65 - Edição Permitida.....	60
Figura 66 - Dados Editados	60
Figura 67 - Preparo de Esconder o Produto (Admin).....	61
Figura 68 - Produto Escondido	61
Figura 69 - Ação da Volta da Visibilidade do Item.....	62
Figura 70 - Eliminar Item	62
Figura 71 - Eliminar Utilizadores	63
Figura 72 - pap_gabi_godeeper_bd: 3.8.1	63
Figura 73 - produtoApi.js: 3.8.2	64
Figura 74 - musicaAdminApi.js: 3.8.3.....	65
Figura 75 - eventoApi.js: 3.8.4	66
Figura 76 - eventoApi.js: 3.8.4 p.2	66
Figura 77 – Problema 1	67
Figura 78 - Resolução1.....	68
Figura 79 – Problema 2	68

Figura 80 – Resolução 2	68
Figura 81 – Problema 3	69
Figura 82 – Resolução 3	69
Figura 83 – Problema 4	70
Figura 84 – Resolução 4	70
Figura 85 – Problema 5	71
Figura 86 – Resolução 5	71
Figura 87 – Problema 6	71
Figura 88 - Resolução6	72

ÍNDICE DE TABELAS

Tabela 1 - Cronograma inicial	16
Tabela 2 - Cronograma final	17

LISTA DE SIGLAS E ACRÓNIMOS

- IDE – Integrated Development Environment
- API – Application Programming Interface (Interface de Programação de Aplicações)
- BD – Base de Dados
- CORS – Cross-Origin Resource Sharing (Partilha de Recursos entre Origens Diferentes)
- CRUD – Create, Read, Update, Delete (Criar, Ler, Atualizar, Eliminar)
- CSS – Cascading Style Sheets (Folhas de Estilo em Cascata)
- HTML – HyperText Markup Language (Linguagem de Marcação de Hipertexto)
- HTTP – HyperText Transfer Protocol (Protocolo de Transferência de Hipertexto)
- IDE – Integrated Development Environment (Ambiente de Desenvolvimento Integrado)
- JSON – JavaScript Object Notation (Notação de Objetos JavaScript)
- JS – JavaScript
- MBWay – MB Way (Serviço de Pagamento Móvel)
- MP3 – MPEG Audio Layer III (Formato de Compressão de Áudio)
- MySQL – My Structured Query Language (Sistema de Gestão de Base de Dados Relacional)
- PAP – Prova de Aptidão Profissional
- PDF – Portable Document Format (Formato de Documento Portátil)
- RESTR – epresentational State Transfer (Transferência de Estado Representacional)
- SQL – Structured Query Language (Linguagem de Consulta Estruturada)
- SVG – Scalable Vector Graphics (Gráficos Vetoriais Escaláveis)
- URL – Uniform Resource Locator (Localizador Uniforme de Recursos)
- VLC – VideoLAN Client (Cliente de Reprodução Multimédia VideoLAN)
- VS Code – Visual Studio Code (Editor de Código Visual Studio)

1. INTRODUÇÃO

A PAP consiste na apresentação e defesa, perante um júri, de um projeto consubstanciado num produto, material ou intelectual, numa intervenção ou numa atuação, consoante a natureza dos cursos, bem como no respetivo relatório final de realização e apreciação crítica, demonstrativo de conhecimentos, aptidões, atitudes e competências profissionais adquiridas ao longo do percurso formativo do aluno, em todas as componentes de formação, com especial enfoque nas áreas de competência inscritas no Perfil dos Alunos à Saída da Escolaridade Obrigatória e no perfil profissional associado à respetiva qualificação.

O presente relatório descreve todas as etapas de conceção e desenvolvimento do projeto GoDeeper, realizado no âmbito da PAP. Está organizado em cinco capítulos: o primeiro é esta introdução, onde se apresenta a fundamentação, a descrição e os objetivos do projeto; o segundo aborda a planificação, incluindo a calendarização, as tecnologias utilizadas, o protótipo e a base de dados; o terceiro descreve a implementação de todas as funcionalidades desenvolvidas; o quarto apresenta os principais problemas encontrados e as soluções adotadas; e o quinto encerra com a conclusão e a apreciação crítica do trabalho realizado.

O projeto enquadra-se no curso de Técnico de Gestão e Programação de Sistemas Informáticos, aplicando conhecimentos de desenvolvimento web, bases de dados relacionais, autenticação e gestão de sistemas, adquiridos ao longo da formação.

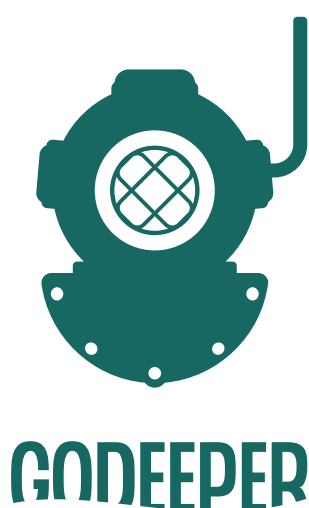


Figura 1 - Logo GoDeeper

1.1. FUNDAMENTAÇÃO

O projeto GoDeeper – Mergulhar na Imersão da Palavra de Deus - nasceu da vontade de divulgar o grupo de jovens da Igreja Baptista de Caldas da Rainha, através de uma plataforma digital moderna e acessível. A ideia surgiu da perceção de que, atualmente, muitos jovens utilizam a internet diariamente para pesquisar, comunicar e aprender. Assim, criar um espaço online dedicado à fé representa uma forma eficaz de aproximar o público jovem de Deus e de promover um relacionamento mais profundo com Ele.

Considero esta ideia boa porque, além de divulgar o grupo de jovens em que participo, o projeto tem como propósito ajudar outras pessoas a conhecer mais sobre Deus e a importância de manter um relacionamento constante com Ele. O GoDeeper procura também mostrar que existe uma igreja aberta ao diálogo, disponível para esclarecer dúvidas sobre a fé e a teologia e disposta a acolher todos aqueles que procuram crescer espiritualmente.

Embora existam outros sites de divulgação de grupos de jovens, o GoDeeper distingue-se por ser um projeto desenvolvido de forma personalizada e adaptada à nossa realidade. O site inclui funcionalidades únicas, como a possibilidade de os utilizadores criarem uma conta com login e password, escolherem as suas músicas preferidas entre as que o grupo canta, comprarem produtos oficiais do grupo e acompanharem eventos e atividades futuras. Além disso, o líder do grupo terá acesso à administração das contas, facilitando a gestão da comunidade e mantendo uma comunicação próxima com todos os membros.

1.2. DESCRIÇÃO DO PROJETO

O GoDeeper é uma aplicação web full-stack desenvolvida com tecnologias modernas, que combina um *backend* em Node.js com Express.js, uma base de dados MySQL e um *frontend* em HTML, CSS com Tailwind CSS e JavaScript. A aplicação inclui ainda um módulo de reprodução de áudio desenvolvido em Python com Flask, que permite a reprodução física de músicas nos dispositivos dos utilizadores.

A plataforma está estruturada em torno de um sistema de contas de utilizador. Qualquer pessoa interessada pode criar uma conta pessoal, inserindo os seus dados pessoais, como nome, apelido, número de telemóvel, endereço de correio eletrónico e a sua nacionalidade. Após o registo, é necessário efetuar o início de sessão com as credenciais escolhidas para aceder à área privada do sítio. Dentro da conta, o utilizador pode ainda terminar a sessão ou eliminar permanentemente o seu perfil.

A plataforma distingue dois tipos de utilizadores: o utilizador comum e o administrador principal (admin_master). O administrador principal tem acesso total à plataforma, incluindo a gestão de todos os conteúdos publicados e o painel de administração de utilizadores.

Depois de iniciar sessão, o utilizador tem acesso a várias funcionalidades:

GoMusic: ouvir as músicas do grupo GoDeeper através de *streaming* de áudio e criar playlists personalizadas com as músicas disponíveis na plataforma e claro com a opção de eliminar a playlist, com controlo de reprodução completo (play, pause, próxima, anterior e volume).

Produtos: consultar e manifestar interesse em artigos oficiais do grupo, como camisolas, moletons, pulseiras, canecas, cadernos de oração e outros produtos personalizados com a identidade visual do GoDeeper. O sistema de contacto para aquisição é feito via MBWay (Apenas o número da pessoa a quem debes enviar a mensagem relativamente ao tamanho, à cor e a outras características do produto.)

Eventos: consultar as atividades e eventos futuros organizados pelo grupo, com filtragem automática por mês e destaque no mês com o evento mais próximo.

Conteúdos e Estudos: visualizar e descarregar materiais de ensino bíblico em formato PDF, que contêm os estudos e temas abordados nos encontros do grupo.

Sobre a Conta: visualizar e editar os dados pessoais da conta, com um modo de edição que pode ser ativado e desativado por um botão de alternância.

Para além destas funcionalidades restritas a utilizadores com conta, a página principal do sítio é acessível publicamente e apresenta informações gerais sobre o grupo GoDeeper, uma secção de líderes, links para os estudos bíblicos e um mapa interativo com as igrejas baptistas do país.

1.3. OBJETIVOS DO PROJETO

O objetivo principal do projeto é divulgar o grupo de jovens da Igreja Baptista de Caldas da Rainha através de uma plataforma digital moderna e acessível, aproximando jovens de Deus e promovendo um relacionamento mais profundo com Ele.

Do ponto de vista funcional, a plataforma permite que os utilizadores criem uma conta pessoal e acedam a conteúdos exclusivos, disponibiliza as músicas do grupo com um sistema de playlists personalizadas, mostra os eventos e atividades futuras, partilha estudos bíblicos e conteúdos em PDF, oferece uma loja de produtos oficiais do grupo e disponibiliza ao líder um painel de administração para gerir a comunidade.

Para além dos objetivos técnicos, o GoDeeper pretende também mostrar que existe uma comunidade de fé aberta ao diálogo, disponível para esclarecer dúvidas sobre a fé e acolher quem procura crescer espiritualmente - fazendo sempre tudo para a glória de Deus.

1.4. ESTADO DO CONHECIMENTO

Para o desenvolvimento do projeto GoDeeper foi necessário adquirir e consolidar conhecimentos em várias áreas da programação e do desenvolvimento web. Uma aplicação com esta natureza envolve não apenas a construção da interface visível ao utilizador, mas também a implementação de lógica de servidor, gestão de dados, autenticação, controlo de acessos, reprodução multimédia e comunicação entre diferentes serviços. Assim, antes do desenvolvimento propriamente dito, foi importante estudar os conceitos fundamentais que sustentam a arquitetura do sistema.

Desenvolvimento web full-stack:

O desenvolvimento web full-stack refere-se à criação integrada das duas principais camadas de uma aplicação web: o *frontend* e o *backend*. O *frontend* corresponde à parte com a qual o utilizador interage diretamente no browser, enquanto o *backend* corresponde à lógica executada no servidor, responsável por processar pedidos, aplicar regras de negócio e comunicar com a base de dados.

No caso do GoDeeper, o *frontend* foi desenvolvido com HTML5, CSS3 e JavaScript. O HTML5 foi utilizado para estruturar o conteúdo das páginas, o CSS3 para a apresentação visual e o JavaScript para introduzir interatividade e permitir comunicação assíncrona com o servidor. Para a estilização da interface foi ainda utilizada a *framework* Tailwind CSS, que facilita a criação de layouts responsivos e consistentes.

O *backend* foi implementado em Node.js, com o *framework* Express.js, que simplifica a criação de servidores web e de APIs. Este componente é responsável por receber os pedidos enviados pelo *frontend*, validar os dados, executar operações sobre a base de dados e devolver respostas adequadas ao cliente.

Base de dados relacional e modelação de dados:

Uma base de dados relacional organiza a informação em tabelas interligadas por relações bem definidas. Este modelo é particularmente útil quando os dados possuem dependências entre si, como acontece no GoDeeper, onde existem entidades como utilizadores, salas, playlists, músicas, produtos, eventos e conteúdos.

Neste projeto foi utilizado MySQL, um sistema de gestão de bases de dados relacionais amplamente usado em aplicações web. A sua utilização exige conhecimentos sobre tabelas, campos, chaves primárias, chaves estrangeiras e integridade referencial. Foi igualmente necessário compreender o conceito de relações entre entidades, por exemplo, um utilizador pode ter várias playlists, uma playlist pode conter várias músicas, e um produto pode ter várias imagens associadas.

Outro conceito importante é o de eliminação em cascata, que permite remover automaticamente registos relacionados quando uma entidade principal é eliminada. Este mecanismo é essencial para manter a consistência da base de dados e evitar dados órfãos.

Api rest e comunicação cliente-servidor:

A comunicação entre o *frontend* e o *backend* foi baseada numa API REST, isto é, numa interface que disponibiliza rotas para operações sobre recursos da aplicação. Este estilo de comunicação segue uma abordagem padronizada e facilita a separação entre apresentação e lógica de negócio.

As operações principais de uma API REST correspondem aos métodos do protocolo HTTP: GET para consultar dados, POST para criar registos, PUT ou PATCH para atualizar informação e DELETE para eliminar recursos. Estas operações estão associadas ao conceito de CRUD: Create, Read, Update e Delete.

No GoDeeper, este modelo é utilizado em várias funcionalidades, como a criação de contas, gestão de playlists, administração de produtos e registo de eventos. A utilização de pedidos HTTP e respostas em JSON permite uma comunicação eficiente entre o browser e o servidor.

Autenticação e gestão de sessões:

A autenticação é o processo de verificação da identidade de um utilizador através das credenciais fornecidas, normalmente nome de utilizador e palavra-passe. Depois de validado o acesso, o sistema precisa de manter o utilizador reconhecido ao longo de vários pedidos, o que é feito através de uma sessão.

Uma sessão permite que o servidor identifique o utilizador sem necessidade de repetir o login em cada página visitada. No GoDeeper, a gestão de sessões foi implementada com a biblioteca express-session, com armazenamento persistente em MySQL. Esta abordagem permite manter a sessão ativa mesmo após reinícios do servidor.

Este tema é particularmente importante porque a aplicação contém áreas protegidas e funcionalidades específicas consoante o perfil do utilizador. Assim, foi necessário compreender bem a relação entre autenticação, sessão e autorização.

Controlo de acessos e perfis de utilizador:

O controlo de acessos define que ações cada tipo de utilizador pode executar dentro da aplicação. Num sistema como o GoDeeper, isto é fundamental para proteger funcionalidades sensíveis e garantir que apenas utilizadores autorizados têm acesso a certas áreas.

Foram considerados dois perfis principais: utilizador e admin_master. O utilizador tem acesso às funcionalidades padrão da aplicação; e o admin_master tem controlo total, incluindo operações sobre utilizadores, conteúdos e configurações gerais.

A proteção das rotas é feita através de middlewares de autenticação e autorização, que verificam se o utilizador possui sessão ativa e se o seu perfil lhe permite aceder ao recurso pedido. Este mecanismo é essencial para assegurar a segurança e a separação de responsabilidades dentro do sistema.

Streaming de áudio e reprodução multimédia:

O *streaming* de áudio é uma técnica que permite reproduzir som enquanto os dados ainda estão a ser transferidos, sem necessidade de descarregar o ficheiro completo antes da reprodução. Esta funcionalidade é especialmente útil em aplicações multimédia, pois melhora a experiência de utilização e reduz o tempo de espera.

No GoDeeper, o *streaming* foi implementado com suporte a Range Requests, uma funcionalidade do protocolo HTTP que permite ao cliente pedir apenas uma parte do ficheiro de cada vez. Isto torna possível controlar a posição de reprodução, avançar ou recuar na música e iniciar o áudio de forma imediata.

Para além da reprodução no browser, o projeto integra ainda um *player* de áudio físico desenvolvido em Python com a biblioteca VLC. Este *player* permite reproduzir música nos altifalantes do dispositivo, funcionando como um serviço independente. Cada utilizador possui um processo próprio associado a uma porta de rede exclusiva, o que garante isolamento entre sessões de áudio.

Gestão de ficheiros e upload:

A gestão de ficheiros é outra área essencial em aplicações web que lidam com multimédia e documentos. O upload consiste na transferência de ficheiros do dispositivo do utilizador para o servidor, permitindo guardar imagens, músicas, PDFs e outros conteúdos associados à aplicação.

No GoDeeper, esta funcionalidade é usada para o envio de imagens de produtos, capas de músicas, ficheiros de áudio e documentos PDF. Para tratar este processo no *backend* foi utilizada a

biblioteca Multer, bastante comum em aplicações Node.js por facilitar o processamento de formulários com ficheiros.

Além do envio, também é necessário compreender a organização e o armazenamento dos ficheiros no sistema, de forma a manter coerência entre os dados guardados na base de dados e os recursos físicos existentes no servidor.

Processos e comunicação entre serviços:

Um processo é um programa em execução no sistema operativo. Em aplicações mais complexas, pode ser necessário iniciar e controlar processos externos para executar tarefas específicas. No GoDeeper, o servidor Node.js tem a responsabilidade de iniciar e gerir processos Python independentes para controlar a reprodução de áudio física.

A comunicação entre o *backend* Node.js e o serviço Python é feita através de pedidos HTTP internos. O Node.js envia comandos como tocar, pausar, parar ou alterar o volume, e o servidor Flask em Python executa essas instruções no *player* VLC. Esta separação permite distribuir responsabilidades entre serviços distintos e torna o sistema mais modular.

A gestão destes processos exige também atenção à sua persistência, exclusividade e sincronização com o estado da aplicação, garantindo que cada utilizador tem apenas o seu *player* associado e que o sistema responde de forma previsível.

Síntese:

Em síntese, o desenvolvimento do GoDeeper exigiu conhecimentos em várias áreas complementares, desde o desenvolvimento web full-stack até à gestão de sessões, controlo de acessos, base de dados, upload de ficheiros e reprodução de áudio. A integração destes conceitos permitiu construir uma aplicação modular, segura e funcional, com separação clara entre *frontend*, *backend*, base de dados e serviços auxiliares.

2. PLANIFICAÇÃO DO PROJETO

2.1. CALENDARIZAÇÃO

A primeira etapa consistiu na elaboração de um cronograma com todas as fases do projeto e respetivas durações tendo em conta a data estipulada para entrega do mesmo.

2.1.1. CALENDARIZAÇÃO PREVISTA

ETAPAS	OUT	NOV	DEZ	JAN	FEV	MAR	ABR	MAI	JUN
Debate de ideias com a turma									
Pesquisa sobre o tema									
Definição dos requisitos e estrutura do projeto									
Desenvolvimento do <i>frontend</i> (HTML/CSS/JS)									
Criação da Base de Dados									
Criação de Conta e Login									
GoMusic - Sistema de músicas e playlists									
Produtos GoDeeper									
Conteúdos GoDeeper									
Painel de Administração									
Elaboração do Relatório									

Tabela 1 - Cronograma inicial

O cronograma inicial foi elaborado com base numa estimativa das principais fases do projeto, distribuídas ao longo do ano letivo. Foram identificadas as etapas fundamentais, desde o debate de ideias até à elaboração do relatório, com especial atenção ao tempo necessário para o desenvolvimento das funcionalidades principais.

2.1.2. CALENDARIZAÇÃO REALIZADA

ETAPAS	OUT	NOV	DEZ	JAN	FEV	MAR	ABR	MAI	JUN
Debate de ideias com a turma									
Pesquisa sobre o tema									
Definição dos requisitos e estrutura do projeto									
Desenvolvimento do <i>frontend</i> (HTML/CSS/JS)									
Desenvolvimento do <i>backend</i> (Node.js)									
Criação da Base de Dados e Finalização									
Criação de Conta e Login									
Edição da Conta e Excluir Conta									
GoMusic - Sistema de músicas e playlists									
Produtos GoDeeper									
Conteúdos GoDeeper									
Painel de Administração – Excluir Conta, Esconder, Adicionar, Excluir: (Música, Produtos, Conteúdos (Excluir Utilizadores))									
Eventos - GoDeeper									

Melhoria do Visual do Site.									
Testes e correção de erros									
Elaboração do Relatório									

Tabela 2 - Cronograma final

Comparando os dois cronogramas, é possível verificar que o desenvolvimento decorreu de forma mais extensa do que o inicialmente previsto. O aumento do número de etapas deveu-se ao crescimento natural do projeto - funcionalidades como a edição de conta, os eventos, as melhorias visuais e os testes não estavam previstas na calendarização inicial, mas foram sendo integradas à medida que o projeto evoluiu. A elaboração do relatório acompanhou o desenvolvimento ao longo de todo o processo.

2.2. TECNOLOGIA UTILIZADA

Este subcapítulo apresenta as tecnologias utilizadas no desenvolvimento do projeto GoDeeper, com a descrição de cada uma, a sua finalidade concreta no projeto e a justificação da escolha face a alternativas possíveis.

Visual Studio Code (VSCode):

É o editor de código onde todo o projeto foi escrito e organizado. Funciona como o "ambiente de trabalho" do programador, com apoio para editar ficheiros, ver erros, pesquisar no projeto e instalar extensões úteis.

Foi o editor utilizado para escrever a totalidade do código do *backend*, *frontend* e do *player* em Python.

Foi escolhido por ser gratuito, leve e por ter um vasto ecossistema de extensões (Tailwind CSS IntelliSense, MySQL, Prettier, Python, entre outras) que aceleram significativamente o desenvolvimento em projetos com múltiplas linguagens, como é o caso do GoDeeper.

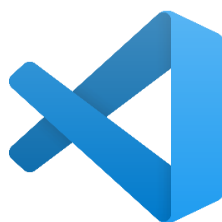


Figura 1 - Logo do Visual Studio Code

HTML - HyperText Markup Language:

É a linguagem que dá estrutura às páginas web, definindo os elementos básicos como títulos, botões, formulários, imagens e secções.

Foi utilizada para estruturar todas as páginas da aplicação, desde a página inicial até ao painel de administração.

É a linguagem de base obrigatória para qualquer página web, sem alternativa real para esta função.



Figura 2 - Logo HTML

Tailwind CSS:

É uma framework de estilos (CSS) que permite dar aspeto visual às páginas de forma rápida e organizada, através de classes utilitárias predefinidas.

Foi utilizada para estilizar toda a interface do site - cores, espaçamentos, alinhamentos e responsividade em diferentes tamanhos de ecrã.

Em vez de escrever CSS manual e repetitivo, o Tailwind permite construir interfaces consistentes de forma mais rápida, o que se revelou especialmente útil num projeto com muitas páginas diferentes desenvolvido por uma só pessoa.



Figura 3 - Logo TailwindCSS

JavaScript:

É a linguagem de programação usada no browser para tornar as páginas interativas.

Serve para responder a cliques, enviar pedidos ao servidor, atualizar conteúdos sem recarregar a página e controlar praticamente todas as ações do utilizador na interface.

É a única linguagem nativamente suportada por todos os browsers para criar interatividade no lado do cliente, tornando-a incontornável para este tipo de aplicação.



Figura 4 - Logo Java Script

Node.js:

É um ambiente que permite executar JavaScript fora do browser, diretamente no servidor.

Foi utilizado para construir toda a lógica principal do *backend* - gestão de pedidos, autenticação, sessões e comunicação com a base de dados.

Permite usar a mesma linguagem (JavaScript) tanto no *frontend* como no *backend*, o que simplifica o desenvolvimento e facilita a partilha de lógica entre as duas partes da aplicação.



Figura 5 - Logo Node.JS

Python:

É uma linguagem de programação simples e versátil.

É utilizada exclusivamente no *player* de áudio - um serviço separado do site que trata da reprodução física da música.

Foi escolhida por ter uma biblioteca madura e fiável para controlo de áudio (VLC), algo que não existe de forma equivalente em Node.js, tornando-a a opção mais adequada para esta funcionalidade específica.



Figura 6 - Logo Python

MySQL:

É um sistema de gestão de base de dados relacional.

Guarda toda a informação da aplicação de forma estruturada, em tabelas ligadas entre si - utilizadores, playlists, músicas, eventos e produtos.

Os dados do GoDeeper têm relações claras entre si (por exemplo, um utilizador tem várias playlists), o que se adequa naturalmente a um modelo relacional; o MySQL foi ainda escolhido por ser gratuito, amplamente documentado e já estudado ao longo do curso.



Figura 7 – Logo MySQL

Express.js:

É uma framework para Node.js que facilita a criação do servidor web.

Ajuda a definir as rotas do servidor, tratar pedidos HTTP, enviar respostas e organizar a lógica do *backend*.

Sem uma framework como o Express, seria necessário escrever manualmente muito código repetitivo para tratar pedidos HTTP; o Express simplifica e organiza esse processo de forma amplamente testada pela comunidade.



Figura 8 - Logo Express JS

Multer:

É uma biblioteca para Node.js que trata uploads de ficheiros.

Permite receber imagens, músicas e PDFs enviados pelos utilizadores ou pelo administrador, e armazená-los no servidor.

É a biblioteca de referência para este tipo de tarefa em projetos Node.js/Express, evitando ter de implementar manualmente a receção e o armazenamento de ficheiros.



Figura 9 - Logo Multer

Flask:

É uma framework web leve em Python.

É utilizada para criar o servidor do *player* de áudio, expondo rotas que o Node.js pode chamar (tocar, pausar, parar).

Por ser leve e simples de configurar, é adequada para um serviço pequeno e específico como este, sem a complexidade de frameworks mais robustas.



Figura 10 - Logo Flask

VLC (VideoLAN Client):

É um software muito conhecido para reprodução de áudio e vídeo.

É usado, através da biblioteca python-vlc, para tocar música fisicamente nos altifalantes do dispositivo.

É uma tecnologia madura, gratuita e amplamente testada para reprodução multimédia, com uma biblioteca Python bem documentada que facilita a integração.

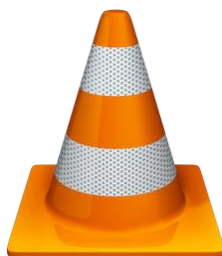


Figura 11 - Logo VLC

Fetch API:

É uma interface do JavaScript que permite enviar pedidos ao servidor e receber respostas sem recarregar a página.

É usada em praticamente todas as interações do site com o servidor - login, reprodução de músicas, gestão de playlists, entre outras.

É uma API nativa dos browsers modernos, o que evita a necessidade de bibliotecas externas apenas para comunicação assíncrona.



Figura 12 - JS Fetch API

JSON (JavaScript Object Notation):

É um formato leve para troca de dados.

É o formato usado em todas as respostas do servidor ao *frontend*, e nos dados guardados na sessão do utilizador.

É o formato padrão para comunicação entre *frontend* e *backend* em aplicações web modernas, sendo diretamente compatível com JavaScript.



Figura 13 - Logo JSON

Claude.ai:

É um assistente de inteligência artificial da Anthropic, acessível através de um navegador web.

Foi utilizado como apoio na resolução de problemas técnicos durante o desenvolvimento, na geração de excertos de código e na revisão de soluções já implementadas.

Foi usado como ferramenta de apoio pontual, semelhante à consulta de documentação técnica ou fóruns de programação, mas nunca substituindo a compreensão e as decisões técnicas tomadas ao longo do projeto, que foram sempre da responsabilidade do autor.



Figura 14 - Logo Claude.ai

2.3. PROTÓTIPO

Antes de iniciar o desenvolvimento de cada página, foi feito um trabalho de planeamento visual através da ferramenta Balsamiq, que permite desenhar wireframes — esquemas simples e rápidos de construir, focados na disposição dos elementos e na estrutura geral de cada ecrã, sem se preocupar ainda com cores, tipografia ou detalhes gráficos finais. Esta etapa foi importante para pensar antecipadamente a organização de cada página, os elementos que deveriam estar presentes e o percurso que o utilizador faria dentro da aplicação, antes de qualquer linha de código ser escrita.

A paleta de cores do projeto foi definida com três cores personalizadas, referenciadas ao longo de todo o código como `goDeeper1`, `goDeeper2` e `goDeeper3`, representando tons de verde azulado e verde escuro que transmitem profundidade, tranquilidade e fé, sendo estas as cores associadas ao grupo. O tipo de letra principal utilizado é o Nunito, uma fonte arredondada e moderna que confere legibilidade e suavidade ao design.

```
}
@theme {
  --color-goDeeper1: #176863;
  --color-goDeeper2: #1e1e1e;
  --color-goDeeper3: #e9f1f4;
  --spacing-120: 30rem;
}
```

Figura 15 - Theme GoDeeper

Com base nestes wireframes, foram planeadas as seguintes páginas principais do projeto.

A página de Login, Boas-Vindas e Registo (correspondente aos ficheiros `GoDeeper-CaldasdaRainha`, `Login-GoDeeper` e `CriarConta-GoDeeper`) foi pensada como uma página pública, com uma secção de boas-vindas onde constam as informações do grupo, o horário habitual dos encontros e um mapa interativo das igrejas baptistas da região, além de permitir ao utilizador autenticar-se ou criar uma nova conta. O formulário de registo foi planeado para incluir os campos de nome, apelido, telemóvel, email, país, nome de utilizador e palavra-passe.

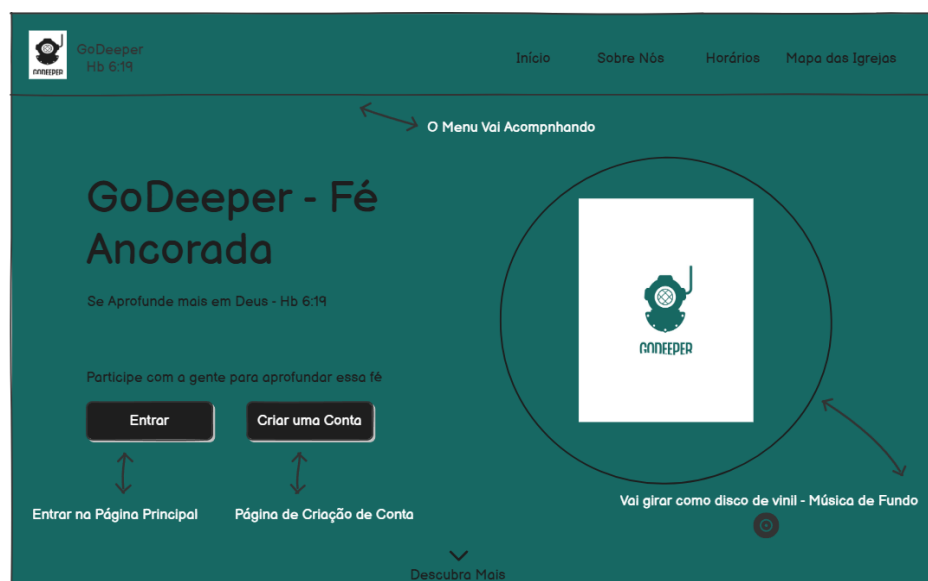


Figura 16 - Página Inicial



Figura 17 - Página Inicial: Sobre Nós



Figura 18 - Mapa Interativo do Site: <https://www.baptistasdocentro.pt/mapa>

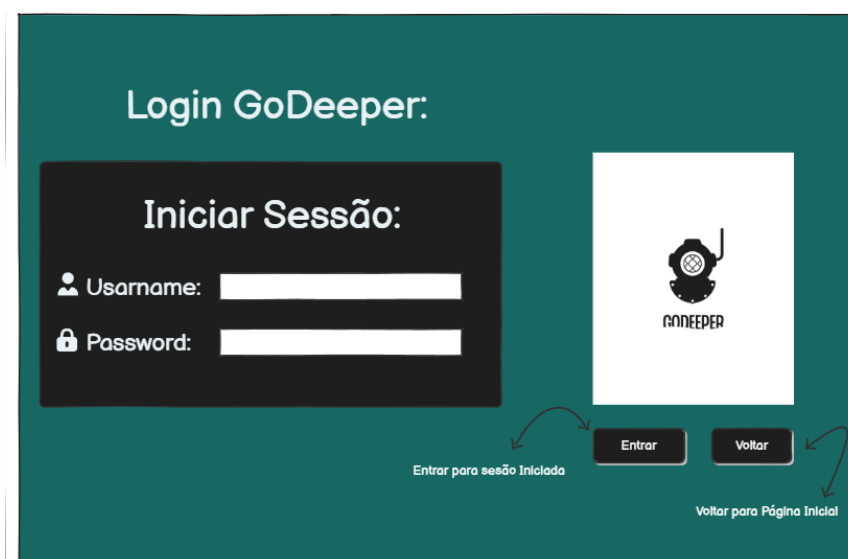


Figura 19 - Página de Login



Figura 20 - Página de Criação de Conta

A Página Principal (GoDeeper) foi planeada como a página acessível após o início de sessão, incluindo secções de apresentação do grupo, dos seus líderes, um mapa de igrejas e um rodapé com as redes sociais do grupo.



Figura 21 - Página Principal



Figura 22 - Página Principal: Liderança



Figura 23 - Página Principal: Explorador de Mapas

A página GoMusic (GoMusic-GoDeeper) foi concebida como a página dedicada ao sistema de músicas, com um menu lateral para gestão de playlists, uma grelha de músicas, secções de músicas em alta e sugestões, e um modal de reprodução com controlos completos.

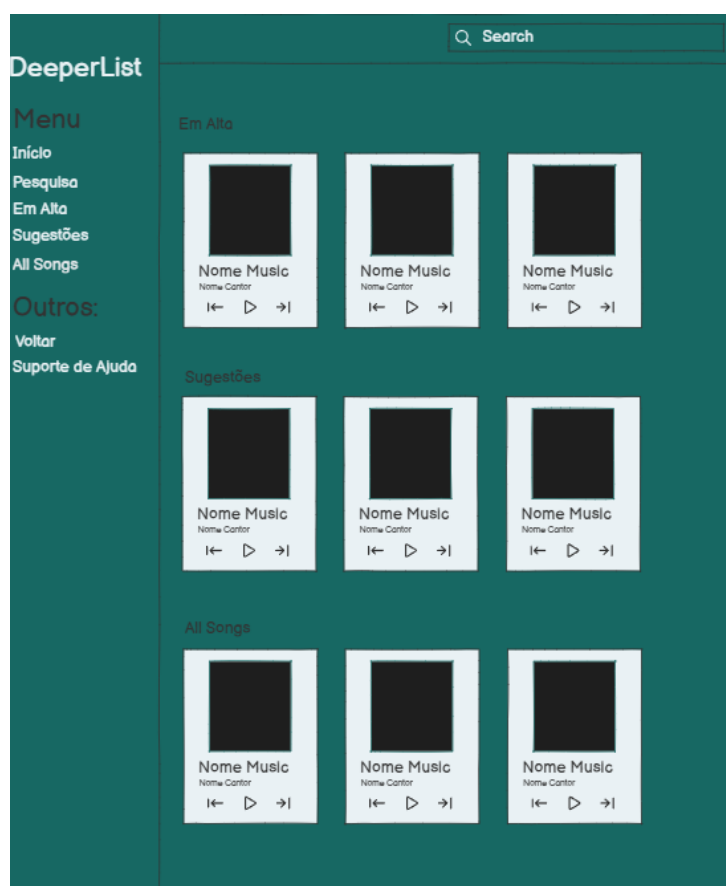


Figura 24 - GoMusic

A página de Produtos (Produtos-GoDeeper) foi planeada com uma grelha de produtos, filtros por categoria, ordenação por preço e por nome, uma barra de pesquisa e um carrossel de imagens em cada produto.

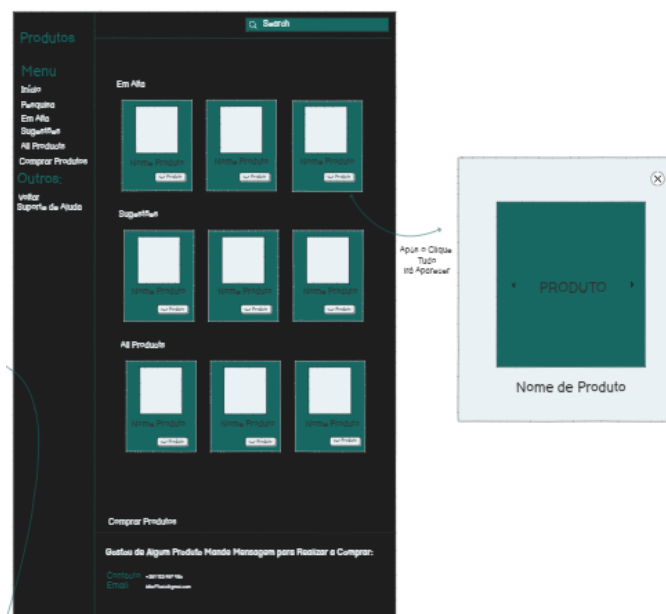


Figura 25 - Produtos

A página Sobre a Conta (Conta-GoDeeper) foi pensada para permitir a visualização e edição dos dados pessoais do utilizador, com um modo de edição ativável através de um botão de alternância.

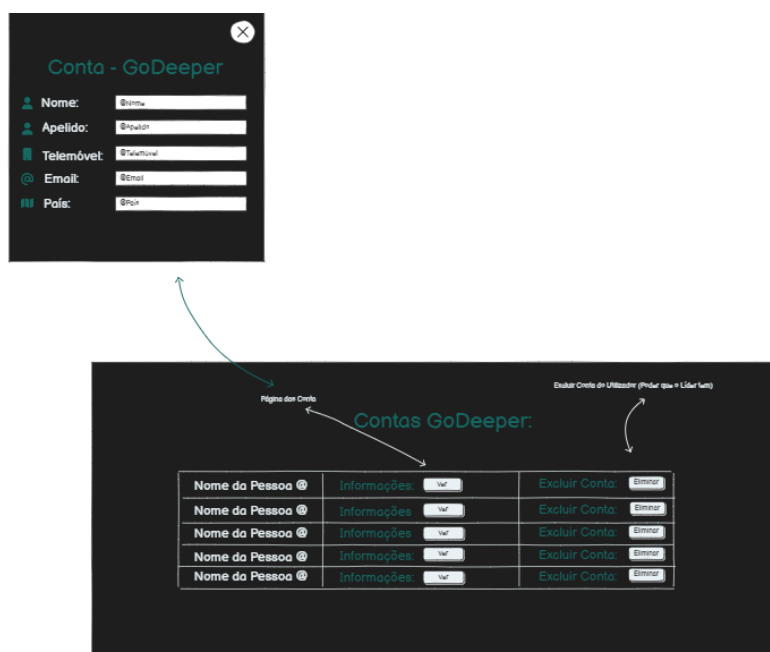


Figura 26 - Sobre a Conta

Por fim, o Painel de Administração (Administracao-GoDeeper) foi planeado como uma página exclusiva da conta admin_master, com uma tabela de todos os utilizadores registados e a opção de eliminação de contas.

Importa referir que este planeamento inicial não incluía ainda todas as páginas que o projeto viria a ter. Nomeadamente, a ideia de reunir os Conteúdos numa página própria e a criação de uma página dedicada aos Eventos não faziam parte da conceção original - surgiram posteriormente, a partir de sugestões dadas pelos professores orientadores ao longo do acompanhamento do projeto, tendo sido incorporadas já numa fase mais avançada do desenvolvimento.

2.4. BASE DE DADOS

A base de dados do projeto GoDeeper foi desenvolvida em MySQL e tem o nome pap_gabi_godeeper_bd. É composta por treze tabelas relacionadas entre si, que suportam todas as funcionalidades da plataforma. Doze dessas tabelas foram criadas diretamente no script SQL do projeto; a décima terceira, chamada sessions, é criada automaticamente pela biblioteca express-mysql-session no primeiro arranque do servidor, sendo responsável por armazenar as sessões ativas dos utilizadores.

Tabela Utilizador:

A tabela Utilizador é a tabela central da base de dados, concentrando os dados de autenticação e identificação de cada conta registada - nome, apelido, contacto, credenciais de acesso e o tipo de utilizador, que distingue uma conta normal de uma conta admin_master, com acesso total à

plataforma. É a partir desta tabela que todos os outros módulos do sistema se relacionam.

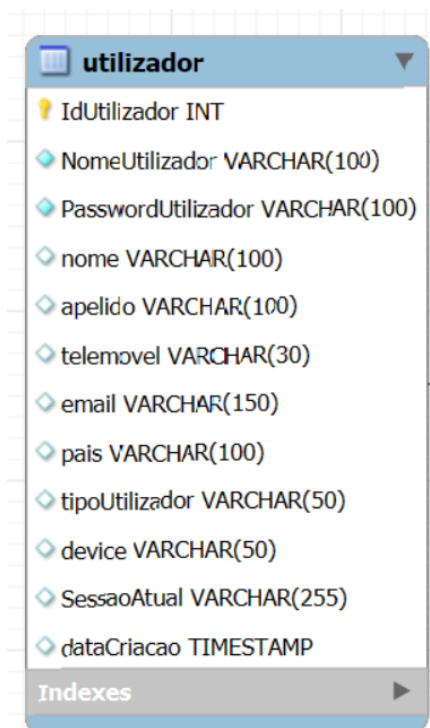


Figura 27 - Base de Dados p.1

Tabela Sala:

A tabela Sala representa o espaço lógico associado a cada utilizador para reprodução de áudio, guardando a porta de rede onde o respetivo processo Python está a funcionar. A relação entre Utilizador e Sala é do tipo 1:1: cada utilizador possui exatamente uma sala, e cada sala pertence a um único utilizador.

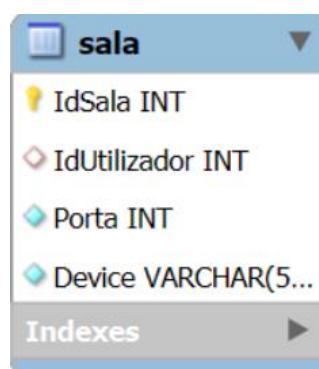


Figura 28 - Base de Dados p.2

Tabela Música:

A tabela Musica guarda todas as músicas disponíveis na plataforma, incluindo o nome, o autor e a localização dos ficheiros de áudio e de imagem. Foi mais tarde alargada com um campo que permite ao admin_master ocultar uma música sem a eliminar definitivamente, e com outro que define em que secção da página essa música deve aparecer.

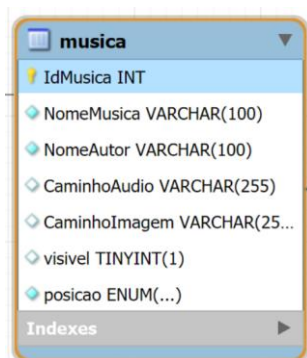


Figura 29 - Base de Dados p.3

Tabelas Playlist e Playlistmusica:

As tabelas Playlist e PlaylistMusica trabalham em conjunto para permitir que cada utilizador crie listas de reprodução personalizadas. A relação entre Utilizador e Playlist é do tipo 1:N - um utilizador pode ter várias playlists - enquanto a relação entre Playlist e Musica é do tipo N:M, uma vez que uma playlist pode conter várias músicas e a mesma música pode pertencer a várias playlists diferentes. É por este motivo que existe a tabela intermédia PlaylistMusica, necessária para representar este tipo de relação numa base de dados relacional. Foi ainda configurada uma regra de eliminação em cascata: se uma música for removida, todas as referências a ela nas playlists são automaticamente eliminadas.

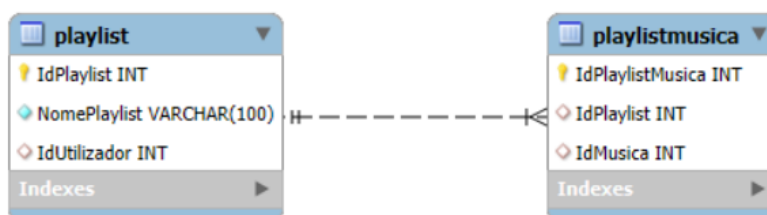


Figura 30 - Base de Dados p.4

Tabela Evento:

A tabela Evento armazena as atividades organizadas pelo grupo GoDeeper, incluindo nome, local, data, hora, descrição e imagem, permitindo que a plataforma mostre aos utilizadores as próximas atividades, organizadas automaticamente por mês.

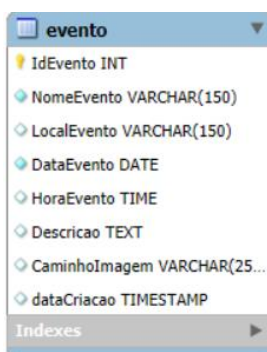


Figura 31 - Base de Dados p.5

Tabelas Conteudo e Conteudopdf:

As tabelas Conteudo e ConteudoPDF suportam a área de estudos bíblicos da plataforma. A relação entre elas é do tipo 1:N - um conteúdo pode ter vários PDFs associados - com eliminação em cascata configurada, de forma que, ao remover um conteúdo, todos os seus PDFs são eliminados automaticamente. À semelhança da tabela Musica, também aqui existe um campo que permite ao admin_master ocultar um conteúdo sem o eliminar.

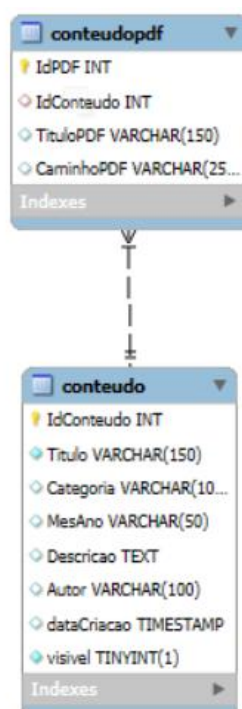


Figura 32 - Base de Dados p.6

Tabelas Produto e Produtoimagem:

As tabelas Produto e ProdutoImagem suportam a loja de produtos oficiais do grupo. A relação entre elas é do tipo 1:N, permitindo que um produto tenha várias imagens associadas e, consequentemente, um carrossel de fotografias na loja.

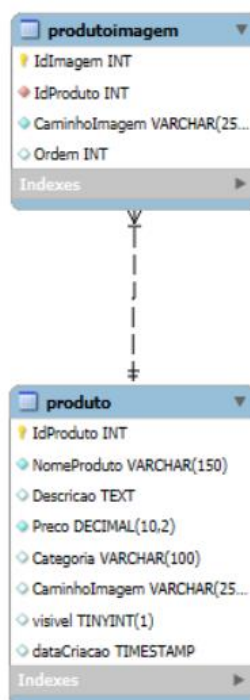


Figura 33 - Base de Dados p.7

Tabelas Logodeeper e Docodeeper:

Por fim, as tabelas LogoGoDeeper e DocGoDeeper armazenam recursos gerais da plataforma - diferentes versões do logótipo e documentos institucionais, como a Declaração de Fé - não estando diretamente associadas a utilizadores.

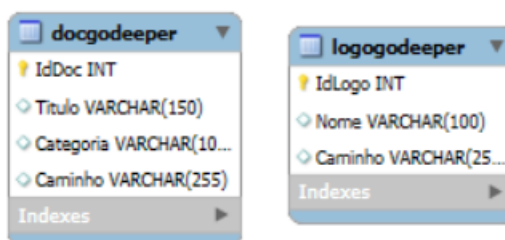


Figura 34 - Base de Dados p.8

Tabela Sessions:

A tabela sessions é a única que não foi criada manualmente através do script SQL do projeto: é gerada automaticamente pela biblioteca express-mysql-session no primeiro arranque do servidor. A sua função é guardar as sessões ativas dos utilizadores diretamente na base de dados MySQL, em vez de apenas na memória do servidor - o que significa que, se o servidor for reiniciado, os utilizadores continuam autenticados, sem precisar de fazer novamente login.

Como se liga ao resto do sistema:

Quando um utilizador faz login com sucesso, o servidor cria um registo nesta tabela com os seus dados de sessão. Em simultâneo, o identificador dessa sessão é também guardado na tabela Utilizador, no campo SessaoAtual. Esta duplicação é intencional: permite detetar situações em que o mesmo utilizador faz login em dois dispositivos diferentes ao mesmo tempo, invalidando a sessão anterior sempre que surge um novo login. A configuração desta tabela é feita no ficheiro server.js, como se observa na Figura 37.

A configuração desta tabela é feita no server.js através das seguintes opções:

```
const MySQLStore = require("express-mysql-session")(session);
const app = express();
const port = 3000;

const sessionStore = new MySQLStore(dbOptions);

app.use(
  session({
    secret: "segredo-super-seguro",
    resave: false,
    saveUninitialized: false,
    store: sessionStore,
    cookie: {
      maxAge: 1000 * 60 * 60 * 24,
      httpOnly: true,
    },
  })
);
```

Figura 35 - server.js: Criação da Tabela: sessions

Relações entre tabelas:

Resumindo as relações mais relevantes da base de dados: Utilizador e Sala relacionam-se de forma 1:1; Utilizador e Playlist relacionam-se de forma 1:N; Playlist e Musica relacionam-se de forma N:M, através da tabela intermédia PlaylistMusica; Conteudo e ConteudoPDF relacionam-se de forma 1:N; e Produto e ProdutoImagem relacionam-se também de forma 1:N. A tabela sessions não possui relações formais por chave estrangeira, mas liga-se indiretamente à tabela Utilizador através do campo SessaoAtual.

No total, a base de dados do GoDeeper é composta por treze tabelas - Utilizador, Sala, Musica, Playlist, PlaylistMusica, Evento, Conteudo, ConteudoPDF, Produto, ProdutoImagem, LogoGoDeeper, DocGoDeeper e sessions - cada uma com uma responsabilidade clara, ligadas entre si através de chaves estrangeiras que garantem a integridade dos dados da plataforma. Em conjunto, suportam todas as funcionalidades do sistema: desde o registo e autenticação de utilizadores, à reprodução de músicas, gestão de playlists, publicação de eventos, conteúdos bíblicos, loja de produtos e controlo de acessos por parte do administrador. A Figura 38 apresenta uma visão geral desta estrutura, com todas as tabelas e respetivas relações.

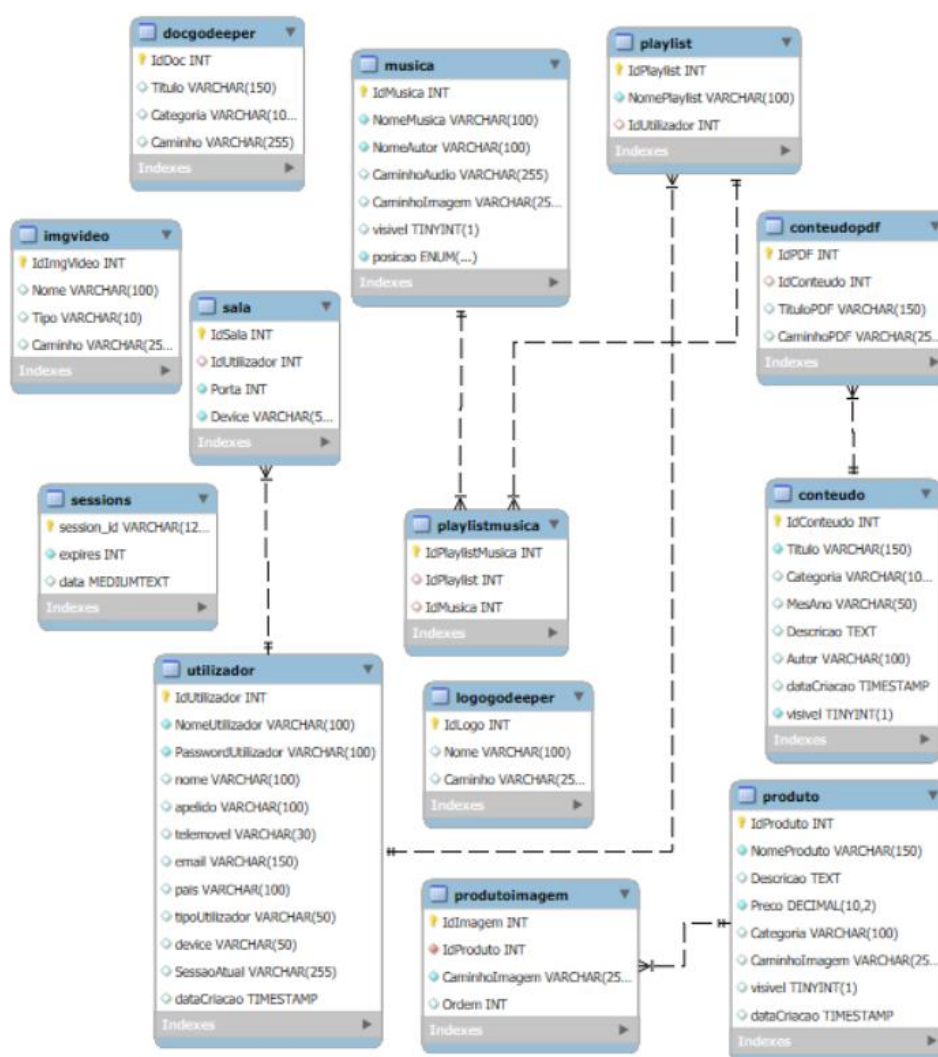


Figura 36 - BD Completa

3. IMPLEMENTAÇÃO DO PROJETO

A sua arquitetura segue o modelo cliente-servidor, onde o *frontend* trata da interface com o utilizador e o *backend* gere a lógica e os dados.

Pode ser entendido como um sistema dividido em quatro componentes principais: *frontend*, *backend*, base de dados e *player* de áudio externo.

O *frontend* corresponde àquilo que o utilizador vê e com que interage. É composto por páginas HTML estilizadas com Tailwind CSS e scripts JavaScript que permitem executar ações como reproduzir músicas, gerir playlists, visualizar eventos ou editar dados pessoais.

O *backend* é um servidor Node.js com Express.js, responsável por receber pedidos HTTP do *frontend*, processá-los, comunicar com a base de dados e devolver respostas em formato JSON. Este servidor corre na porta 3000 e funciona como o núcleo central da aplicação.

A base de dados MySQL armazena toda a informação do sistema, incluindo utilizadores, músicas, playlists, eventos, produtos e conteúdos. As relações entre entidades são estruturadas, por exemplo: um utilizador possui uma sala e várias playlists, enquanto uma playlist pode conter várias músicas.

Por fim, o sistema inclui um componente externo em Python: um servidor Flask que utiliza a biblioteca VLC para reproduzir áudio diretamente nos altifalantes do dispositivo. Este componente funciona como um *player* físico independente.

De forma resumida, o sistema está dividido em quatro grandes componentes: o *frontend* (páginas HTML, CSS e JavaScript), o *backend* (servidor Node.js com Express.js), a base de dados (MySQL) e o *player* de áudio (um serviço externo escrito em Python). Estes quatro componentes comunicam entre si através de pedidos HTTP, trocando informação no formato JSON.

3.1 - Comunicação entre Componentes:

A comunicação entre o *frontend* e o *backend* é feita através de pedidos HTTP, utilizando a Fetch API do JavaScript. Sempre que o utilizador realiza uma ação - como clicar em "Tocar", guardar uma playlist ou editar os seus dados - o *frontend* envia um pedido para uma rota específica do servidor. O servidor processa esse pedido, comunica com a base de dados (e, quando necessário, com o *player* Python) e devolve uma resposta em formato JSON, que o *frontend* utiliza para atualizar a página sem necessidade de a recarregar.

Um exemplo simples deste fluxo é o botão de reproduzir uma música: o JavaScript do *frontend* envia um pedido POST para a rota `/tocarSala`, incluindo a sala do utilizador e o caminho

do ficheiro de áudio. O *backend* recebe esse pedido, identifica a porta associada à sala do utilizador e reencaminha o pedido para o *player* Python correspondente, devolvendo depois uma resposta de sucesso ou de erro ao *frontend*.

3.2 - Estrutura e Implementação:

O projeto está organizado em pastas que separam claramente as responsabilidades de cada componente, o que facilita a manutenção e a leitura do código. As pastas principais são:

- *src/backend* - contém toda a lógica do servidor Node.js: rotas, middlewares e configuração da base de dados.
- *src/soundDevices* - contém o sistema de reprodução de áudio em Python.
- *frontend* - contém as páginas HTML, os ficheiros CSS e os scripts JavaScript visíveis ao utilizador.
- *fileSystem* - pasta onde são guardados os ficheiros enviados para o servidor, como músicas, imagens e documentos PDF.

O servidor é iniciado através do ficheiro *server.js*, onde são configurados os middlewares, rotas e a porta de execução.

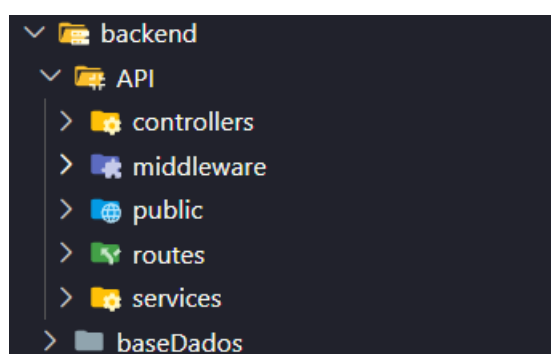


Figura 37 - Organização das Pastas p.1

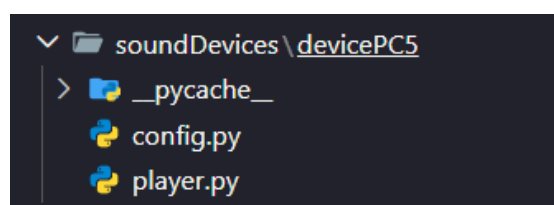


Figura 38 - Organização das Pastas p.2

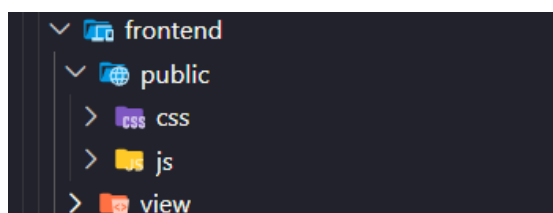


Figura 39 - Organização das Pastas p.3

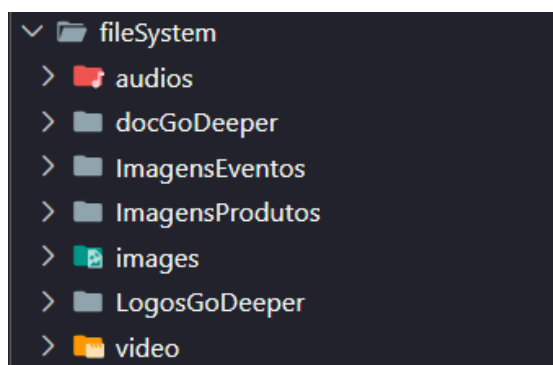


Figura 40 - Organização das Pastas p.4

Extensões Instaladas no Visual Studio Code:

Para além do editor em si, foram instaladas diversas extensões no Visual Studio Code que facilitaram e agilizaram o desenvolvimento do projeto, quer ao nível da escrita de código, quer ao nível da organização e legibilidade do mesmo. Estas extensões foram escolhidas em função das tecnologias utilizadas no GoDeeper - Tailwind CSS, Python, MySQL - bem como de necessidades práticas que foram surgindo ao longo do desenvolvimento, como a visualização de ficheiros PDF ou a formatação automática de código. Segue-se uma breve descrição de cada uma delas e da sua utilidade no contexto deste projeto.

Tailwind CSS IntelliSense - sugere automaticamente as classes do Tailwind CSS enquanto se escreve o código, evitando erros e acelerando o desenvolvimento da interface.

Vscode-pdf - permite visualizar ficheiros PDF diretamente no editor, sem precisar de abrir outro programa.

Python - adiciona suporte à linguagem Python no VSCode, incluindo realce de sintaxe e deteção de erros, essencial para desenvolver o *player* de áudio.

Prettier - Code Formatter - formata o código automaticamente, mantendo uma estrutura limpa e consistente em todos os ficheiros do projeto.

MySQL - permite ligar ao servidor de base de dados e executar queries SQL diretamente dentro do VSCode, sem sair do editor.

Material Icon Theme - substitui os ícones das pastas e ficheiros por ícones visuais mais intuitivos, facilitando a navegação entre os ficheiros do projeto.

Live Server - inicia um servidor local com atualização automática da página no browser sempre que o código é alterado, útil para testar o *frontend* rapidamente.

Image Preview - mostra uma pré-visualização das imagens referenciadas no código, diretamente na margem do editor.

Dracula Theme Official - tema de cores escuro aplicado ao editor, tornando a leitura do código mais confortável durante longas sessões de trabalho.

Configuração do Ambiente

Antes de iniciar o projeto pela primeira vez, é necessário instalar todas as dependências e configurar os dois ambientes de execução: o servidor Node.js e o *player* de áudio Python.

1. Configuração do Servidor Node.js:

Passo 1 - Inicialização do projeto Node (apenas na primeira vez):

```
cd projetoFinal
```

```
npm init -y
```

Passo 2 - Instalação das bibliotecas necessárias:

```
npm install body-parser@2.2.0 express-mysql-session@3.0.3 express-session@1.18.1  
express@5.1.0 mysql2@3.14.1 node-fetch@2.7.0 tailwindcss@4.1.8
```

Passo 3 - Instalação do Multer (gestão de uploads de ficheiros):

```
npm install multer
```

Passo 4 - Compilação do CSS do Tailwind (mantém a atualização automática):

```
npx @tailwindcss/cli -i ./src/frontend/public/css/input.css -o  
./src/frontend/public/css/output.css --watch
```

Ou em alternativa, através do script definido no package.json:

```
npm run dev
```

2. Configuração do Player de Áudio Python:

Passo 1 - Criação do ambiente virtual (apenas na primeira vez):

```
cd projetoFinal
python -m venv venv
```

Passo 2 - Ativação do ambiente virtual:

```
venv\Scripts\activate
```

Passo 3 - Instalação das bibliotecas Python necessárias:

```
pip install flask python-vlc flask-cors
```

3. Inicialização do Projeto (Após todas as Instalações):

Para iniciar o CSS (Tailwind):

```
cd projetoFinal
npm run dev
```

Para iniciar o servidor Node.js:

```
cd projetoFinal/src/backend/api/public
node server.js
```

Nota: o *player* de áudio Python é iniciado automaticamente pelo servidor Node.js no momento em que cada utilizador faz login, não sendo necessário iniciá-lo manualmente.

4. Comandos Úteis Adicionais:

Verificação das bibliotecas Node instaladas no projeto:

```
npm list
```

Verificação das bibliotecas Python instaladas (com o venv ativo):

```
pip list
```

3.3 Autenticação e Sessões:

O sistema de autenticação utiliza express-session com armazenamento em MySQL, garantindo persistência mesmo após reinício do servidor.

O processo de autenticação inicia-se quando o utilizador envia as suas credenciais de acesso para o servidor. De seguida, o servidor valida essas credenciais através da consulta à base de dados. Se forem válidas, é criada uma sessão com os dados do utilizador, que fica armazenada e associada à sua conta, permitindo manter a autenticação durante a utilização da plataforma.

Para proteger as rotas privadas, foi criado um middleware de autenticação que verifica se existe uma sessão ativa antes de permitir o acesso. No caso das funcionalidades exclusivas do administrador, é feita uma verificação adicional ao campo tipoUtilizador, garantindo que só a conta admin_master pode aceder ao painel de administração.

No *frontend*, existe uma verificação automática da validade da sessão a cada cinco segundos, como se observa na Figura 41. Caso a sessão tenha expirado ou tenha sido encerrada noutro local, o utilizador é imediatamente redirecionado para a página de login, evitando que continue a utilizar uma área que já não lhe pertence. Esta verificação devolve também o tipo de utilizador, permitindo ao *frontend* mostrar ou esconder funcionalidades de administração de forma dinâmica.

```
Resposta da verificação de sessão: playlist.js:2006
▶ {valido: true, tipoUtilizador: 'utilizador'}
Resposta da verificação de sessão: playlist.js:2006
▶ {valido: true, tipoUtilizador: 'utilizador'}
Resposta da verificação de sessão: playlist.js:2006
▶ {valido: true, tipoUtilizador: 'utilizador'}
```

Figura 41 - Verificação da Sessão a Cada 5 segundos

Quando um novo utilizador cria conta, como ilustrado na Figura 42, o sistema cria automaticamente a sua sala e atribui-lhe uma porta de rede única, preparando desde logo o ambiente de reprodução de áudio. O resultado deste processo pode ser observado no terminal do servidor, na Figura 43, onde é possível acompanhar em tempo real cada passo da criação da conta.

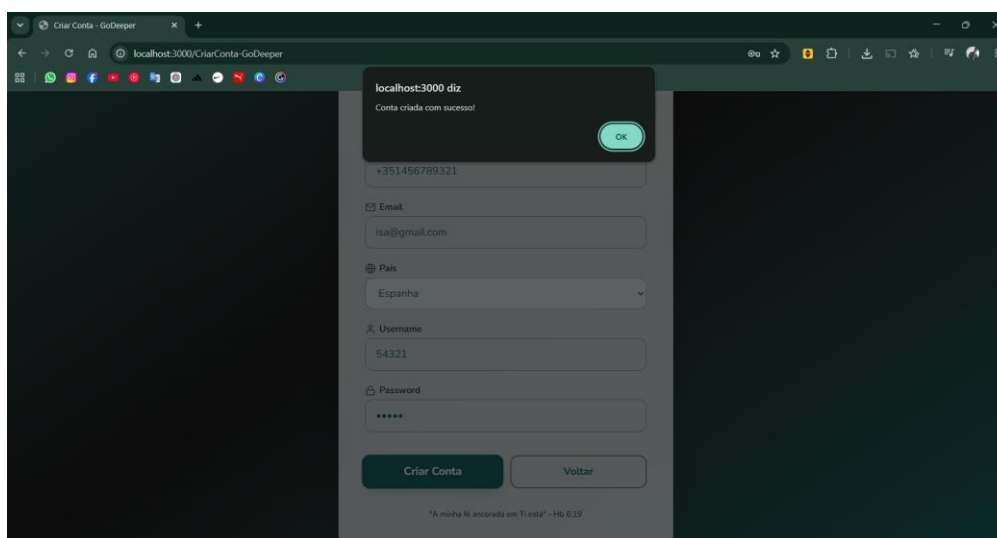


Figura 42 - Criação de Conta

```
{
  nome: 'Isabel',
  apelido: 'Bispo',
  telemovel: '+351456789321',
  email: 'isa@gmail.com',
  pais: 'Espanha',
  NomeUtilizador: '54321',
  PasswordUtilizador: '54321'
}
[LOGIN] Utilizador 54321 → porta 4004 | Device: devicePC5
[PLAYER] Caminho do player.py: C:\Users\gmbis\Desktop\PAP\PAPGabiv.3\projetoFinal\src\soundDevices\devicePC5\player.py
[PLAYER] Python path: C:\Users\gmbis\Desktop\PAP\PAPGabiv.3\projetoFinal\venv\Scripts\python.exe
[PLAYER] A iniciar na porta 4004...
[PLAYER] Player iniciado com sucesso na porta 4004
[PLAYER] A iniciar na porta 4004...
[PLAYER] Servidor Flask ativo na porta 4004
* Serving Flask app 'player'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:4004
Press CTRL+C to quit
█
```

Figura 43 - Informações do Terminal da Criação da Conta

Um dos pontos importantes do sistema de autenticação deste projeto é a forma como são tratadas as sessões que já estavam ativas antes do reinício do servidor. Quando o servidor Node.js é iniciado, este consulta a base de dados para verificar quais os utilizadores que ainda tinham uma sessão ativa. Depois dessa verificação, o sistema inicia automaticamente o processo Python associado a cada utilizador, permitindo que o *player* de áudio continue disponível sem ser necessário realizar novamente o login.

Sem este processo, sempre que o servidor fosse reiniciado, os utilizadores que já estavam autenticados perderiam o acesso ao *player* até iniciarem uma nova sessão. Esta funcionalidade está implementada no ficheiro *server.js*, apresentado na Figura 44, e o seu funcionamento pode ser observado no terminal durante o arranque do projeto, como apresentado na Figura 45.

```
// --- Ao arrancar, inicia players para sessões já ativas ---
app.listen(port, () => {
  console.log(`Servidor a correr em http://localhost:${port}`);

  db.query(
    `SELECT u.NomeUtilizador, s.Porta
    FROM Utilizador u
    INNER JOIN Sala s ON s.IdUtilizador = u.IdUtilizador
    WHERE u.SessaoAtual IS NOT NULL`,
    (err, results) => {
      if (err) {
        console.error("[SERVER] Erro ao buscar sessões ativas:", err);
        return;
      }

      if (results.length === 0) {
        console.log("[SERVER] Nenhuma sessão ativa encontrada.");
        return;
      }

      console.log(
        `[SERVER] ${results.length} sessão(ões) ativa(s) - a iniciar players...`,
      );

      results.forEach((user) => {
        console.log(
          `[SERVER] A iniciar player para ${user.NomeUtilizador} na porta ${user.Porta}`,
        );
        iniciarPlayerPython(user.Porta);
      });
    },
  );
});
});
```

Figura 44- server.js

É o coração do servidor. Ao arrancar, verifica automaticamente quem já tinha sessão ativa e inicia os *players* Python correspondentes.

Inicializando o Projeto:

```
comandos-terminal.txt
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

Figura 45 - Terminal: Iniciar Projeto

O login do utilizador também está ligado ao início do *player* de áudio. Quando um utilizador entra na sua conta pela primeira vez, o servidor inicia automaticamente o processo Python responsável pelo seu *player* de áudio. Esse processo fica associado à porta de rede atribuída ao utilizador, permitindo que o *player* funcione corretamente

Esta funcionalidade está implementada no ficheiro Login-GoDeeper.js, apresentado na Figura 48. Na prática, depois de o login ser concluído com sucesso e a sessão ser criada, o sistema executa uma função que inicia o *player* Python correspondente ao utilizador.


```

if (salaResults.length > 0) {
  const { Porta, Device } = salaResults[0];
  console.log(`[LOGIN] Utilizador ${user.NomeUtilizador} → porta ${Porta} | Device: ${Device}`);

  // Inicia o player para TODOS (admins e utilizadores)
  iniciarPlayerPython(Porta);
} else {
  console.warn(`[LOGIN] Nenhuma sala encontrada para ${user.NomeUtilizador}`);
}

// Guarda o tipoUtilizador na sessão
req.session.utilizador = {
  id: user.IdUtilizador,
  nome: user.NomeUtilizador,
  tipoUtilizador: user.tipoUtilizador,
};

```

Figura 46 - Login-GoDeeper.js

Liga o início de sessão ao arranque automático do *player* Python do utilizador. De forma semelhante, ao criar uma conta pela primeira vez, o sistema não se limita a guardar os dados do novo utilizador - cria também automaticamente a sua "sala" e atribui-lhe uma porta de rede exclusiva, preparando-o desde logo para poder utilizar o *player* de áudio assim que fizer login. Este processo é feito de forma sequencial: primeiro é inserido o novo utilizador na base de dados, depois é consultada a última porta já atribuída a outros utilizadores, e por fim é criada a sua sala com a porta seguinte disponível, evitando que duas contas fiquem associadas à mesma porta. Esta lógica está implementada no ficheiro `apiCriarConta.js`, apresentado na Figura 47.

```

const [result] = await connection.execute(sqlUtilizador, values);
const idNovoUtilizador = result.insertId;

// — 2. Descobrir a próxima porta disponível para o devicePC5 —
const [rows] = await connection.execute(
  `SELECT MAX(Porta) AS ultimaPorta FROM Sala WHERE Device = 'devicePC5'`
);

const ultimaPorta = rows[0].ultimaPorta;
const novaPorta = ultimaPorta ? ultimaPorta + 1 : PORTA_INICIAL_USERS;

// — 3. Criar a sala com a porta e device atribuídos —
await connection.execute(
  `INSERT INTO Sala (IdUtilizador, Porta, Device) VALUES (?, ?, 'devicePC5')`,
  [idNovoUtilizador, novaPorta]
);

await connection.end();

res.json({
  success: true,
  message: "Conta criada com sucesso",
});
} catch (error) {
  console.log(error);

  res.status(500).json({
    success: false,
    message: error.message,
  });
}
});

module.exports = router;

```

Figura 47 - `apiCriarConta.js`

Automatiza a criação da sala e atribuição de uma porta única a cada novo utilizador.

Por fim, foi criada uma rota que permite ao *frontend* verificar regularmente se a sessão do utilizador continua ativa e qual o tipo de conta que está autenticada. Esta verificação é feita automaticamente pelo browser a cada cinco segundos. A rota devolve a informação sobre a validade da sessão e o tipo de utilizador, permitindo que a interface mostre ou esconda funcionalidades específicas, como as opções disponíveis apenas para o administrador. Desta forma, o utilizador não precisa de atualizar a página nem voltar a fazer login. Esta funcionalidade está implementada no ficheiro `verificarSessao.js`, apresentado na Figura 48.

```
return res.json({
  valido,
  tipoUtilizador: valido ? results[0].tipoUtilizador : null,
});
```

Figura 48 - `verificarSessao.js`

Devolve o tipo de utilizador, permitindo ao *frontend* mostrar ou esconder funcionalidades de administração dinamicamente.

3.4. Sistema de Músicas:

A reprodução de música no GoDeeper funciona em duas camadas complementares: a reprodução no navegador (streaming) e a reprodução física através de um processo Python independente.

Streaming no navegador

O servidor Node.js envia os ficheiros de áudio de forma progressiva, recorrendo a um mecanismo do protocolo HTTP designado por Range Requests. Este mecanismo permite que o navegador peça apenas uma parte do ficheiro de cada vez, em vez de ter de esperar que o ficheiro seja transferido na totalidade antes de começar a tocar. É exatamente o princípio utilizado em serviços como o YouTube ou o Spotify: a reprodução começa de imediato e o resto do ficheiro vai sendo carregado enquanto a música é ouvida. Se o utilizador avançar para o meio da música, o servidor envia apenas essa parte, sem recomençar do início.

Reprodução física (*player* Python)

Cada utilizador tem associada uma "sala" na base de dados, à qual corresponde uma porta de rede própria. Quando o utilizador inicia sessão, o servidor Node.js arranca automaticamente um processo Python nessa porta. Este processo cria um pequeno servidor Flask que expõe rotas como

/tocarSala, /pausarSala, /pararSala, /aumentarVolume, /diminuirVolume e /estadoMusica, e que utiliza a biblioteca VLC para reproduzir o áudio diretamente nos altifalantes do computador.

- /tocarSala → Dá o Play na Música.
- /pausarSala → A Música é Pausada.
- /pararSala → A Música é Parada.
- /aumentarVolume → O Volume é Aumentado.
- /diminuirVolume → O Volume é Diminuído
- /estadoMusica → O Estado da Música: Está Parada?? Ou Retomada.

O controlo destes processos é feito por um gestor central (*playerManager.js*) que evita duplicações e garante que cada utilizador tem exatamente um *player* ativo na sua porta. Como cada utilizador tem a sua própria porta, é possível existirem várias pessoas a ouvir músicas diferentes em simultâneo, sem que uma reprodução interfira com a outra.

```

PROBLEMAS  SAÍDA  CONSOLA DE DEPURACÃO  TERMINAL  PORTAS

Press CTRL+C to quit
[PLAYER 4804] Pedido para tocar: http://localhost:3000/stream-audio/musica2.mp3
[PLAYER 4804] Música iniciada: http://localhost:3000/stream-audio/musica2.mp3
127.0.0.1 - - [15/Jun/2026 22:30:36] "POST /tocarSala HTTP/1.1" 200 -
[PLAYER 4804] Pedido para parar
[PLAYER 4804] Música parada
127.0.0.1 - - [15/Jun/2026 22:30:42] "POST /pararSala HTTP/1.1" 200 -
[PLAYER 4804] Pedido para parar
[PLAYER 4804] Música parada
127.0.0.1 - - [15/Jun/2026 22:30:51] "POST /pararSala HTTP/1.1" 200 -
[PLAYER 4804] Pedido para tocar: http://localhost:3000/stream-audio/musica2.mp3
[PLAYER 4804] Música iniciada: http://localhost:3000/stream-audio/musica2.mp3
127.0.0.1 - - [15/Jun/2026 22:32:07] "POST /tocarSala HTTP/1.1" 200 -
[PLAYER 4804] Estado atual: a tocar
127.0.0.1 - - [15/Jun/2026 22:32:09] "GET /estadoMusica HTTP/1.1" 200 -
[PLAYER 4804] Estado atual: a tocar
127.0.0.1 - - [15/Jun/2026 22:32:12] "GET /estadoMusica HTTP/1.1" 200 -
[PLAYER 4804] Pedido para pausar/retomar
[PLAYER 4804] Música pausada
127.0.0.1 - - [15/Jun/2026 22:32:15] "POST /pausarSala HTTP/1.1" 200 -
[PLAYER 4804] Estado atual: parado
127.0.0.1 - - [15/Jun/2026 22:32:15] "GET /estadoMusica HTTP/1.1" 200 -
[PLAYER 4804] Estado atual: parado
127.0.0.1 - - [15/Jun/2026 22:32:18] "GET /estadoMusica HTTP/1.1" 200 -
[PLAYER 4804] Pedido para tocar: http://localhost:3000/stream-audio/musica2.mp3
[PLAYER 4804] Música retomada
127.0.0.1 - - [15/Jun/2026 22:32:20] "POST /tocarSala HTTP/1.1" 200 -
[PLAYER 4804] Estado atual: a tocar
127.0.0.1 - - [15/Jun/2026 22:32:21] "GET /estadoMusica HTTP/1.1" 200 -
[PLAYER 4804] Estado atual: a tocar
127.0.0.1 - - [15/Jun/2026 22:32:24] "GET /estadoMusica HTTP/1.1" 200 -
[PLAYER 4804] Pedido para tocar: http://localhost:3000/stream-audio/musica2.mp3
[PLAYER 4804] Já está a tocar
127.0.0.1 - - [15/Jun/2026 22:32:24] "POST /tocarSala HTTP/1.1" 200 -
[PLAYER 4804] Estado atual: a tocar
127.0.0.1 - - [15/Jun/2026 22:32:27] "GET /estadoMusica HTTP/1.1" 200 -
[PLAYER 4804] Pedido para parar

```

Figura 49 - Informação do Terminal da Ação da Playlist

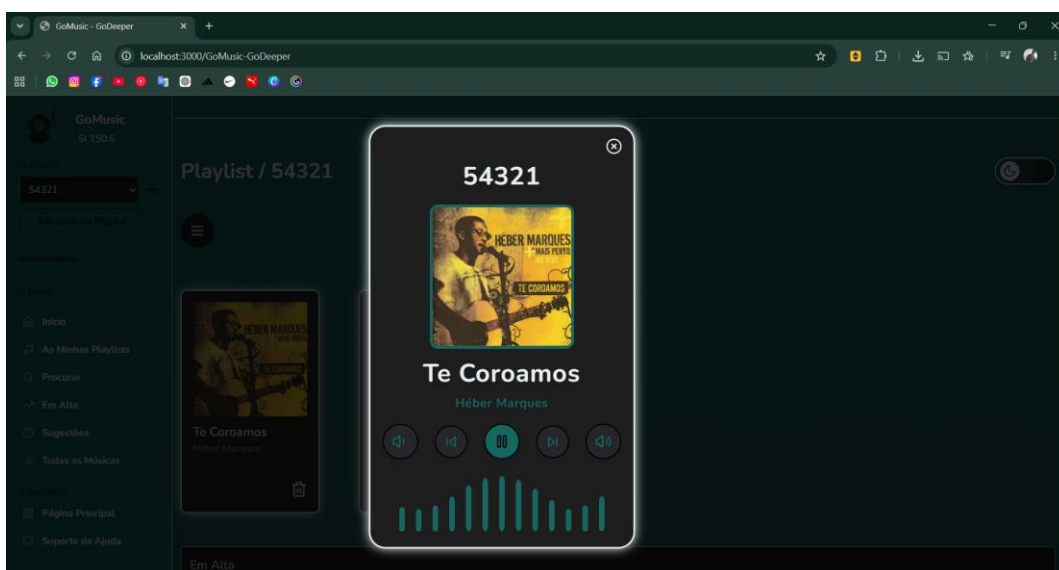


Figura 50 – Playlist

3.5. Sistema de playlists:

O sistema de playlists permite que cada utilizador crie listas de reprodução próprias, compostas pelas músicas disponíveis na plataforma. As suas funcionalidades principais passam pela criação de playlists, pela adição e remoção de músicas dentro de cada uma, e pela reprodução sequencial das músicas que a compõem.

A reprodução sequencial é controlada no *frontend* por um Web Worker que verifica periodicamente o estado da música atual; quando esta termina, o sistema avança automaticamente para a música seguinte da lista, sem necessidade de intervenção do utilizador.

Na Figura 51 pode observar-se o momento de criação de uma nova playlist, onde o utilizador atribui um nome à lista antes de a poder preencher com músicas. Após criada, a playlist pode ser aberta e consultada, como mostra a Figura 52, sendo possível ver todas as músicas atualmente adicionadas a essa lista.

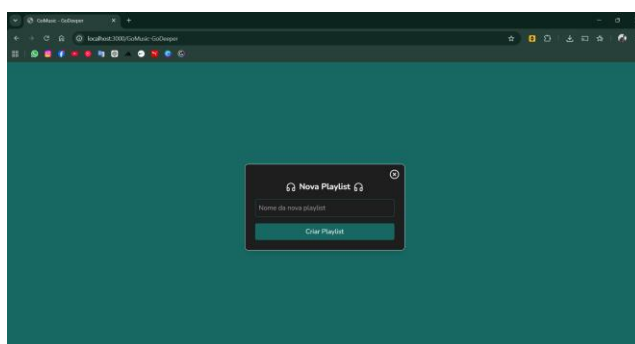


Figura 51 - Criação de Playlist

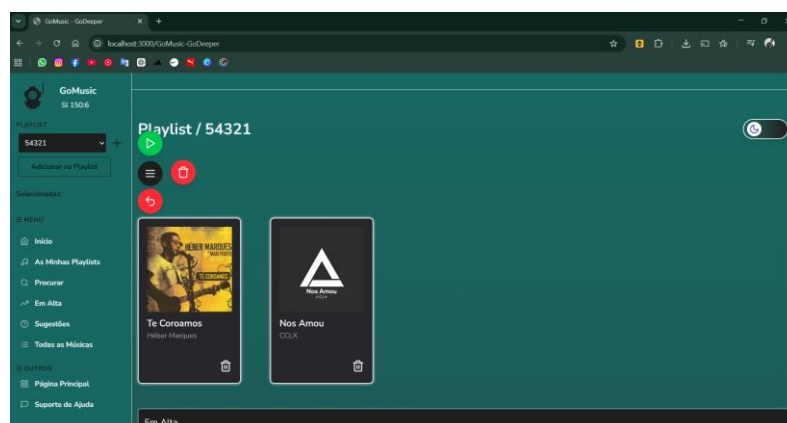


Figura 52 - Playlist Aberta

Caso o utilizador pretenda apagar uma playlist que já não necessite, essa ação está disponível diretamente na interface, como se observa na Figura 53. De forma semelhante, é também possível remover apenas uma música específica de dentro de uma playlist, sem eliminar a lista completa, conforme ilustrado na Figura 54.

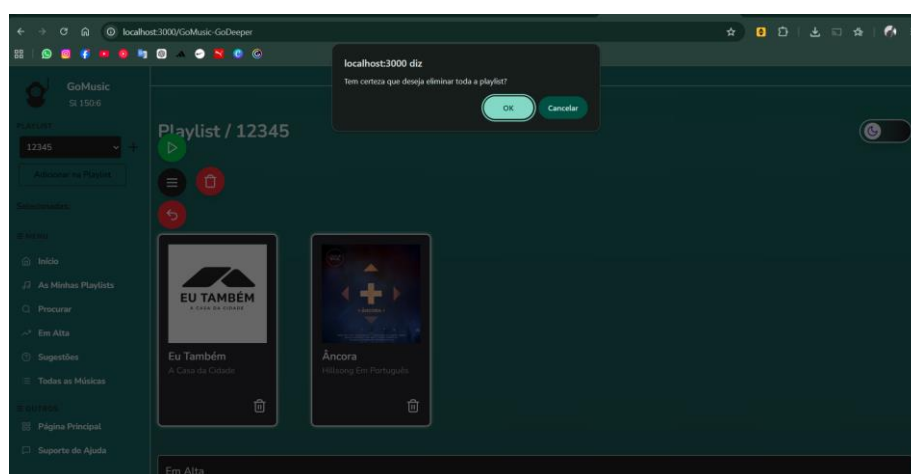


Figura 53 - Excluir Playlist

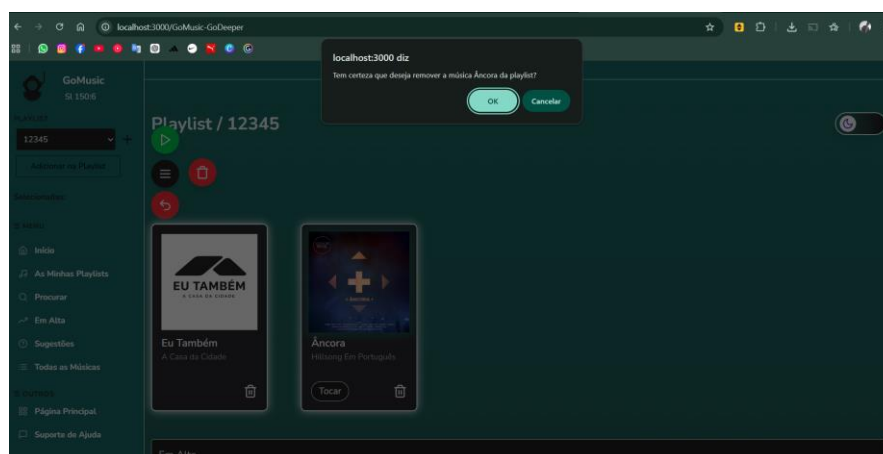


Figura 54 - Remover Música da Playlist

Por fim, a Figura 55 apresenta as diferentes ações disponíveis sobre cada música apresentada na plataforma, nomeadamente a possibilidade de a selecionar para adicionar a uma playlist existente.

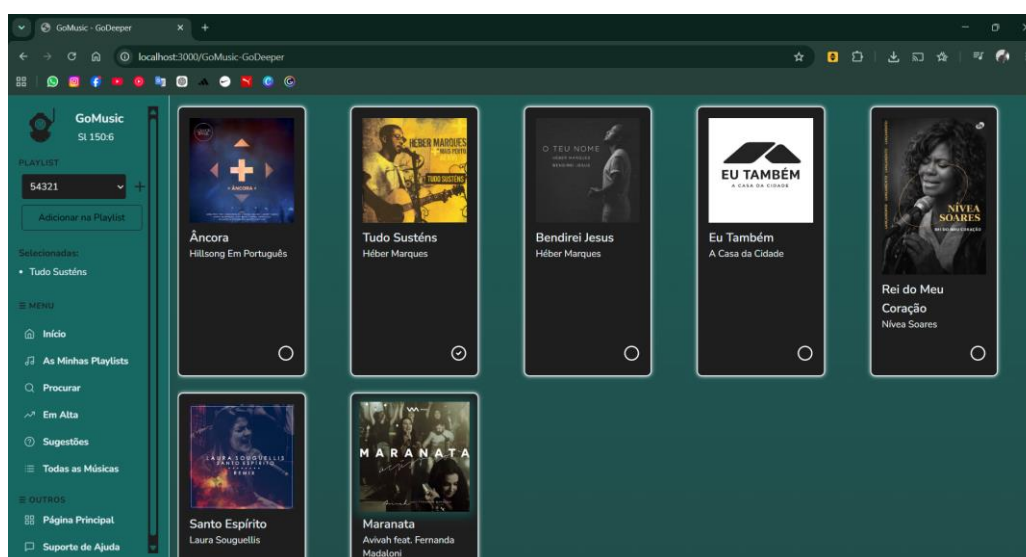


Figura 55 - Ações da Escolha das Músicas

Para que cada utilizador consiga ouvir música de forma independente, sem interferir com a reprodução de outros, é necessário garantir que existe sempre exatamente um processo Python ativo por utilizador - nunca dois, nem nenhum. Se dois processos fossem abertos na mesma porta por engano, ocorreria um erro de conflito; se nenhum fosse aberto, o utilizador simplesmente não conseguiria tocar música. Para resolver isto, foi criado um ficheiro responsável apenas por gerir

estes processos, guardando registo de quais já estão ativos e em que porta, evitando duplicações.

Esta lógica está implementada no ficheiro *playerManager.js*, apresentado na Figura 56.

```
function iniciarPlayerPython(porta) {
  if (playersAtivos[porta]) {
    console.log(`[PLAYER] Já ativo na porta ${porta}`);
    return;
  }

  console.log(`[PLAYER] Caminho do player.py: ${playerPath}`);
  console.log(`[PLAYER] Python path: ${pythonPath}`);
  console.log(`[PLAYER] A iniciar na porta ${porta}...`);

  const processo = spawn(pythonPath, [playerPath, porta], {
    detached: false,
    stdio: "inherit",
  });

  processo.on("error", (err) => {
    console.error(`[PLAYER] Erro ao iniciar player na porta ${porta}:`, err.message);
    delete playersAtivos[porta];
  });

  processo.on("close", (code) => {
    console.log(`[PLAYER] Player na porta ${porta} terminou com código ${code}`);
    delete playersAtivos[porta];
  });

  playersAtivos[porta] = processo;
  console.log(`[PLAYER] Player iniciado com sucesso na porta ${porta}`);
}

module.exports = { iniciarPlayerPython };
```

Figura 56 - *playerManager.js*

Garante que cada utilizador tem exatamente um *player* ativo na sua porta, sem duplicações.

Outro aspeto importante da reprodução de música é a forma como o ficheiro de áudio chega até ao browser do utilizador. Em vez de o servidor enviar o ficheiro completo de uma só vez - o que obrigaria a esperar que a música terminasse de descarregar antes de começar a tocar o servidor envia o áudio por partes, à medida que é necessário. É exatamente o mesmo princípio usado em serviços como o YouTube ou o Spotify: a música começa a tocar quase de imediato, e o resto vai sendo carregado enquanto se ouve. Se o utilizador avançar para o meio da música, o servidor envia apenas essa parte, sem ter de recomençar o envio desde o início. O contrário seria semelhante aos downloads antigos, em que era preciso esperar que o ficheiro ficasse 100% descarregado antes de o poder abrir. Esta técnica, conhecida por Range Requests, está implementada no ficheiro *musicas.js*, apresentado na Figura 57.

```

if (range) {
  const parts = range.replace(/bytes=/, "").split("-");
  const start = parseInt(parts[0], 10);
  const end = parts[1] ? parseInt(parts[1], 10) : fileSize - 1;
  const chunkSize = end - start + 1;

  const file = fs.createReadStream(filePath, { start, end });

  const head = {
    "Content-Range": `bytes ${start}-${end}/${fileSize}`,
    "Accept-Ranges": "bytes",
    "Content-Length": chunkSize,
    "Content-Type": "audio/mpeg",
  };

  res.writeHead(206, head);
  file.pipe(res);
} else {
  const head = {
    "Content-Length": fileSize,
    "Content-Type": "audio/mpeg",
  };
}

```

Figura 57 - musicas.js

Do lado do Python, foi criado um pequeno servidor que funciona como o "motor" da reprodução física de áudio. Este programa recebe comandos do Node.js e traduz-nos em ações reais sobre o som, como tocar, pausar, parar ou ajustar o volume, comunicando diretamente com a biblioteca VLC para produzir o som nos altifalantes do computador.

Um aspeto importante deste programa é que ele não tem uma porta fixa "escrita" no código. Em vez disso, a porta em que deve funcionar é-lhe indicada no momento em que é iniciado, como um argumento - é como dizer ao programa "arranca, mas fica à escuta especificamente nesta porta". Isto é o que permite que cada utilizador tenha a sua própria instância deste programa a correr de forma completamente independente das restantes, cada uma na sua porta, sem que uma interfira com a outra. Este comportamento está implementado no ficheiro *player.py*, apresentado nas Figuras 58 e 59.


```

@app.route('/tocarSala', methods=['POST'])
def play_music():
    global player, musica_atual, estado_musica

    data = request.get_json()
    caminho_audio = data.get('CaminhoAudio')

    print(f"[PLAYER {port}] Pedido para tocar: {caminho_audio}")

    if not caminho_audio:
        return jsonify({'success': False, 'message': 'Falta CaminhoAudio'}), 400

    try:
        if musica_atual == caminho_audio:
            estado = player.get_state() if player else None
            if estado == vlc.State.Paused:
                player.play()
                estado_musica = "a_tocar"
                print(f"[PLAYER {port}] Música retomada")
                return jsonify({'success': True, 'message': 'Música retomada'})
            elif estado == vlc.State.Playing:
                print(f"[PLAYER {port}] Já está a tocar")
                return jsonify({'success': True, 'message': 'Já está a tocar'})
            player.stop()

        if player:
            player.stop()

        player = vlc.MediaPlayer(caminho_audio)
        musica_atual = caminho_audio
        player.play()
        estado_musica = "a_tocar"
        print(f"[PLAYER {port}] Música iniciada: {caminho_audio}")

        Thread(target=monitorar_musica, daemon=True).start()

        return jsonify({'success': True, 'message': 'Música iniciada'})
    except Exception as e:
        print(f"[PLAYER {port}] Erro ao tocar: {str(e)}")
        return jsonify({'success': False, 'message': str(e)}), 500

```

Figura 58 - player.py

```

if __name__ == '__main__':
    print(f"[PLAYER] Servidor Flask ativo na porta {port}")
    app.run(host='127.0.0.1', port=port)

```

Figura 59 - player.py

Como referido anteriormente, a comunicação entre o Node.js e o Python é feita através de pedidos HTTP, exatamente da mesma forma que o browser comunica com o servidor. Falta agora perceber, em concreto, como isso está implementado no código.

Sempre que o utilizador carrega em "Tocar" numa música, o Node.js recebe esse pedido, mas ainda não sabe para onde o deve reencaminhar - precisa primeiro de descobrir em que porta está a funcionar o processo Python daquele utilizador específico. Para isso, consulta a base de dados, pesquisando pela sala do utilizador e obtendo a porta que lhe está associada. Só depois de ter essa informação é que o Node.js envia um novo pedido HTTP, desta vez diretamente para essa porta,

indicando qual o ficheiro de áudio a reproduzir. É este pedido que o Python recebe e transforma em som real através do VLC, como explicado no ponto anterior.

Este processo - consultar primeiro a porta na base de dados e só depois enviar o comando para essa porta específica - está implementado no ficheiro tocarSala.js, apresentado na Figura 60.

```
try {
  // — Busca a porta da sala na BD —
  const connection = await mysql.createConnection(dbConfig);

  const [rows] = await connection.execute(
    "SELECT Porta FROM Sala WHERE IdSala = ?",
    [idSala]
  );

  await connection.end();

  if (rows.length === 0) {
    return res
      .status(404)
      .json({ success: false, message: "Sala não encontrada" });
  }

  const portDevice = rows[0].Porta;

  // — Envia pedido ao dispositivo —
  const deviceResponse = await fetch(
    `http://localhost:${portDevice}/tocarSala`,
    {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ CaminhoAudio }),
    }
  );
}
```

Figura 60 - tocarSala.js

Esta figura mostra o momento exato em que o Node.js consulta a base de dados para obter a porta do utilizador e, de seguida, envia o pedido HTTP para essa porta, reencaminhando o comando de reprodução para o processo Python correto.

3.6. Módulos da Aplicação:

Eventos - Godeeper

Os eventos organizados pelo grupo são apresentados agrupados por mês, mostrando apenas os eventos cuja data ainda não passou. O mês com o evento mais próximo é automaticamente destacado visualmente. O administrador pode criar novos eventos através de um formulário (nome, local, data, hora e descrição) e remover eventos existentes, existindo também uma validação que impede a criação de eventos com uma data anterior ao dia atual. Após o término do horário do evento, este deixa automaticamente de ser apresentado aos utilizadores.

Figura 61 - Criação de Eventos – Administração

Conteúdos Bíblicos – Godeeper:

Este módulo permite ao administrador publicar estudos bíblicos, cada um podendo ter vários documentos PDF associados. Ao criar um conteúdo, é possível adicionar mais do que um ficheiro PDF de uma só vez, através de um botão que vai gerando novos campos de upload conforme necessário. Quando um conteúdo é eliminado, todos os PDFs associados são também eliminados, tanto da base de dados como do sistema de ficheiros.

Figura 62 - Criação de Conteúdo – Administração

Produtos – Godeeper:

A loja de produtos apresenta artigos oficiais do grupo, organizados por categoria e com a possibilidade de cada produto ter várias imagens, apresentadas num carrossel. Como a plataforma não processa pagamentos online, o contacto para aquisição de um produto é feito através de

MBWay: ao manifestar interesse, é mostrado ao utilizador um número de telefone para o qual deve enviar uma mensagem com o tamanho, a cor ou outras características pretendidas. O administrador pode adicionar, esconder ou remover produtos

Figura 63 - Criação de Produtos - Administração

Figura 64 - Interesse: Número para o Pedido

Sobre a Conta - Godeeper:

Nesta área, o utilizador pode consultar os seus dados pessoais e ativar um modo de edição através de um botão de alternância ("toggle"). Enquanto a edição estiver desativada, os campos do formulário aparecem bloqueados, servindo apenas para consulta; ao ativar a edição, os campos tornam-se editáveis e surge a opção de guardar as alterações. Se o utilizador desativar a edição sem guardar, os valores originais são automaticamente repostos.

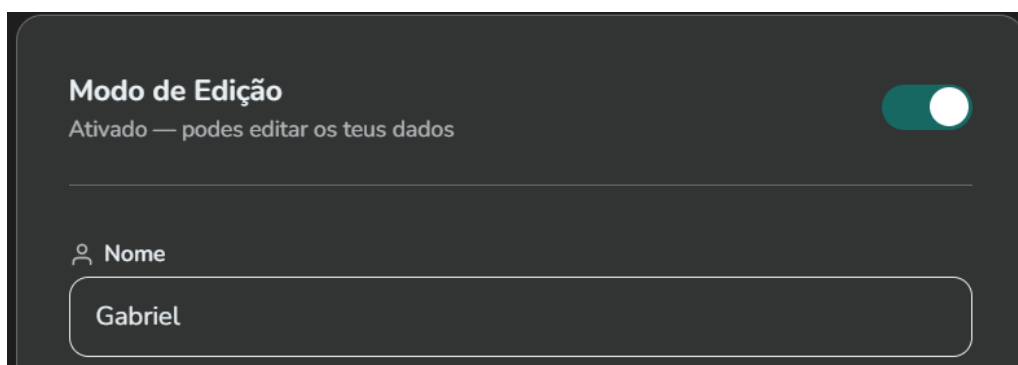


Figura 65 - Edição Permitida

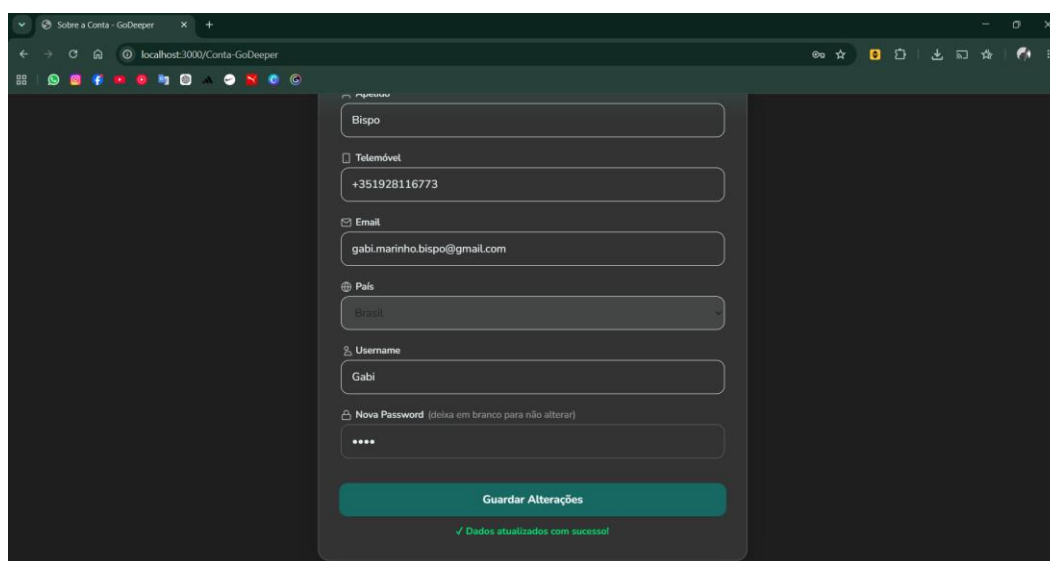


Figura 66 - Dados Editados

3.7 Painel de Administração:

A conta com o tipo de utilizador `admin_master` tem acesso a um conjunto de funcionalidades que não estão disponíveis às contas normais, nomeadamente a gestão de músicas, de eventos, de conteúdos, de produtos e de utilizadores.

Estas funcionalidades incluem três tipos de ações distintas sobre cada elemento: adicionar, esconder (ocultar temporariamente) e remover (eliminar permanentemente).

A eliminação de uma conta de utilizador é particularmente sensível, porque essa conta está ligada a outras tabelas da base de dados. Por esse motivo, a eliminação é feita numa ordem específica, respeitando as dependências entre tabelas, de forma a não violar as relações entre chaves estrangeiras:

PlaylistMusica → Playlist → Sala → Utilizador

Ou seja, antes de eliminar o próprio utilizador, o sistema elimina primeiro as músicas das suas playlists, depois as playlists e, por fim, a sua sala. Só depois destes passos é eliminado o registo do utilizador.

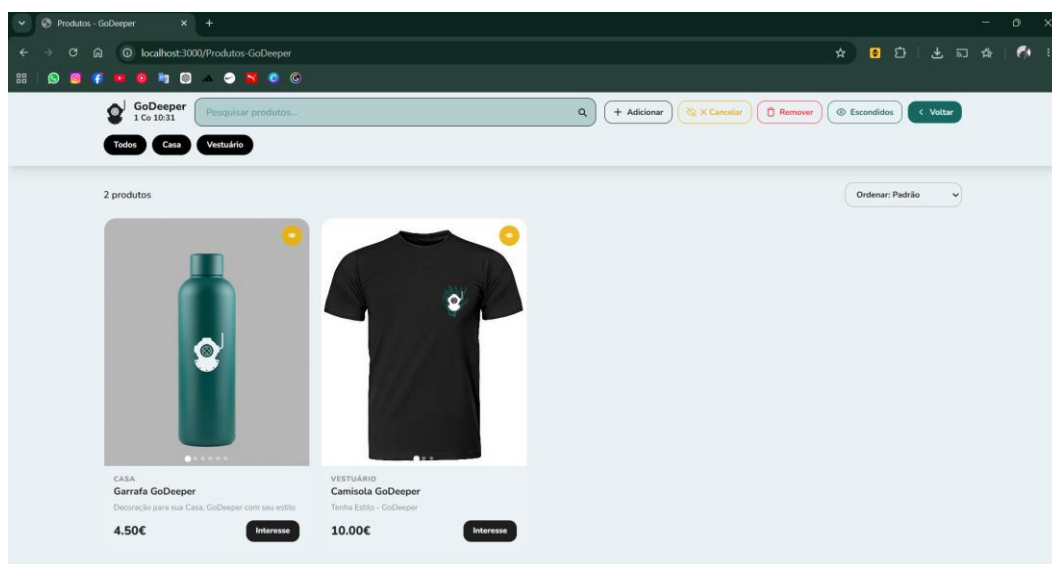


Figura 67 - Preparo de Esconder o Produto (Admin)

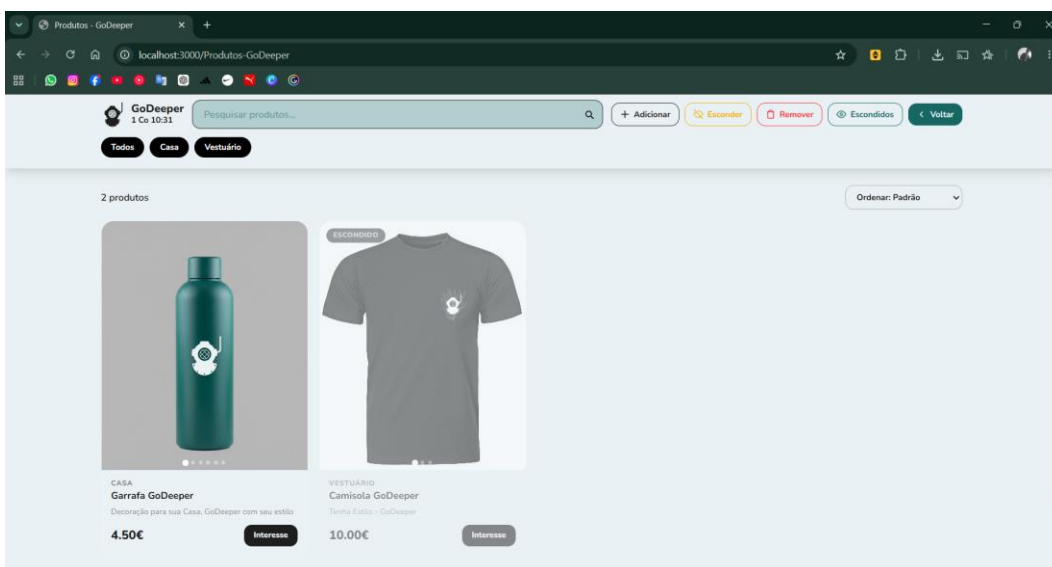


Figura 68 - Produto Escondido

A opção de ocultar o item faz com que este fique invisível para os utilizadores comuns. O item não é eliminado, mas os utilizadores deixam de lhe ter acesso, uma vez que permanece oculto.

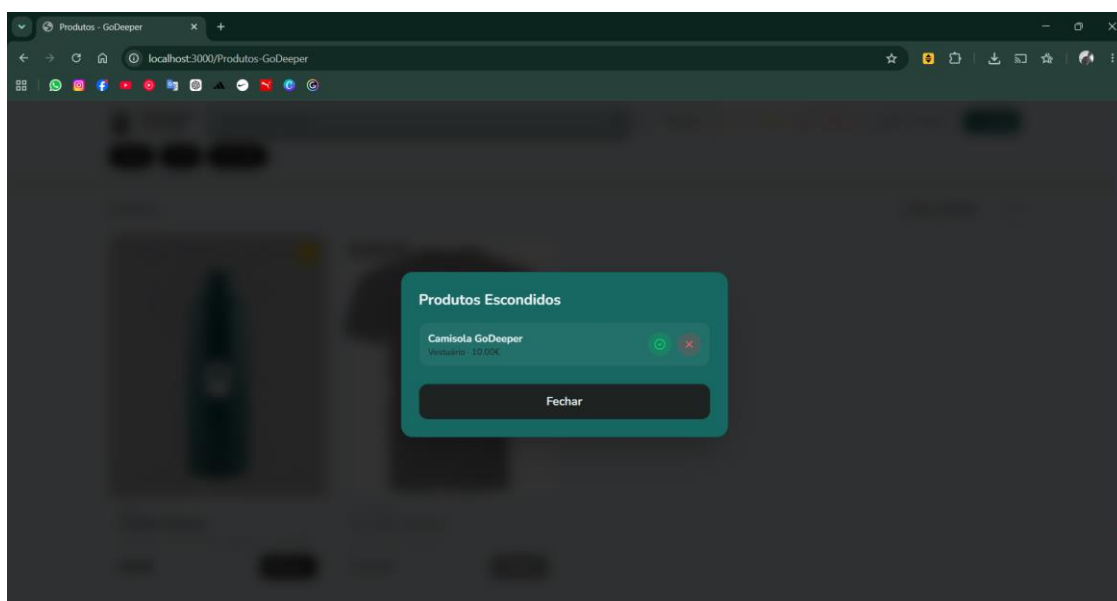


Figura 69 - Ação da Volta da Visibilidade do Item

O administrador tem a opção de voltar a tornar o item visível ou de o eliminar.

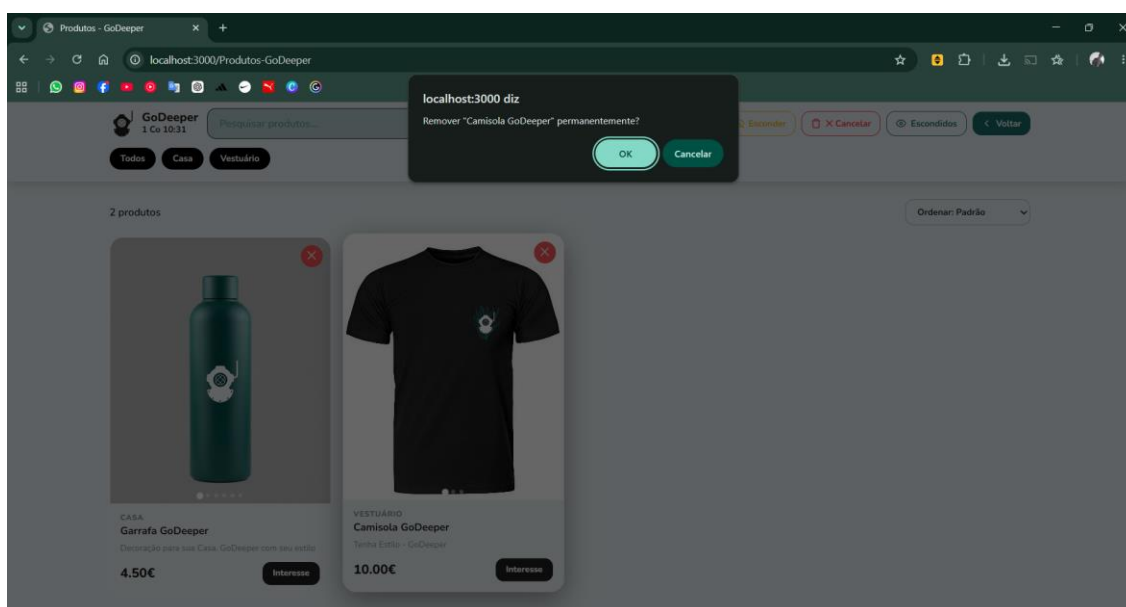


Figura 70 - Eliminar Item

O painel de administração apresenta todos os utilizadores e permite ações diretas sobre cada conta.

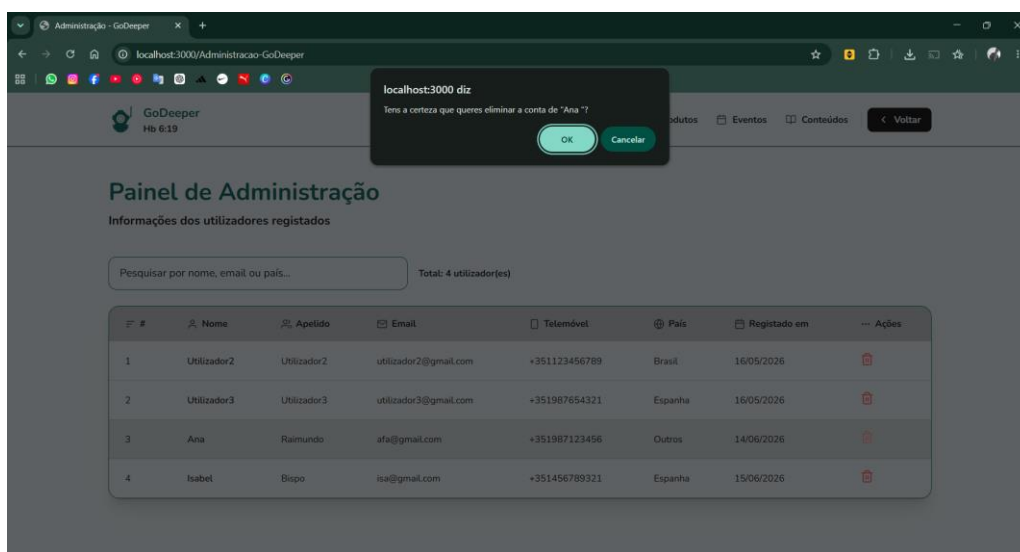


Figura 71 - Eliminar Utilizadores

3.8.1. A coluna "visível" na base de dados:

Para tornar isto possível, foi adicionada a cada uma das tabelas relevantes (Musica, Conteúdo e Produto) uma nova coluna chamada visível, do tipo TINYINT(1). Esta coluna funciona como um interruptor: o valor 1 significa que o elemento está visível para todos os utilizadores, e o valor 0 significa que está escondido, sendo visível apenas pelo administrador. Por defeito, todos os elementos novos são criados com o valor 1.

É importante sublinhar que esconder um elemento não o remove da base de dados: apenas altera o valor desta coluna. É essa a principal diferença entre "esconder" e "eliminar", e é também o que permite que a ação seja facilmente revertida.

```
ALTER TABLE Conteudo ADD COLUMN visivel TINYINT(1) NOT NULL DEFAULT 1;

-- Garante que todos os conteúdos existentes ficam visíveis
UPDATE Conteudo SET visivel = 1;
```

Figura 72 - pap_gabi_godeeper_bd: 3.8.1

3.8.2. Filtragem consoante o tipo de utilizador

A diferença entre o que um utilizador comum vê e o que o administrador vê está na consulta (query) feita à base de dados. Sempre que a lista de músicas, conteúdos ou produtos é pedida ao servidor, o *backend* verifica primeiro o tipo de utilizador guardado na sessão. O administrador (admin_master) vê todos os elementos, incluindo os escondidos; os restantes utilizadores só veem os elementos com visível = 1.

Este mesmo princípio - mudar a condição WHERE consoante o tipo de utilizador - é aplicado nas rotas de listagem de Musica, Conteudo e Produto. Desta forma, o administrador consegue sempre ver e gerir os elementos escondidos, enquanto os utilizadores comuns nunca chegam a saber que eles existem.

```
// — GET /produtoApi/lista —
router.get("/lista", apenasAutenticado, async (req, res) => {
  try {
    const isAdmin = req.session.utilizador.tipoUtilizador === "admin_master";
    const sql = isAdmin
      ? "SELECT * FROM Produto ORDER BY Categoria, NomeProduto ASC"
      : "SELECT * FROM Produto WHERE visivel = 1 ORDER BY Categoria, NomeProduto ASC";

    const [produtos] = await db.query(sql);

    // Para cada produto, busca as imagens
    for (const p of produtos) {
      const [imagens] = await db.query(
        "SELECT CaminhoImagem FROM ProdutoImagem WHERE IdProduto = ? ORDER BY Ordem ASC",
        [p.IdProduto]
      );
      p.imagens = imagens.map(i => i.CaminhoImagem);
    }

    res.json({ success: true, produtos });
  } catch (erro) {
    console.error("Erro ao buscar produtos:", erro);
    res.status(500).json({ success: false, message: "Erro interno" });
  }
});
```

Figura 73 - produtoApi.js: 3.8.2

3.8.3. As rotas de esconder e mostrar:

Para alternar o valor da coluna visivel, foram criadas rotas específicas no *backend*, do tipo PATCH (um método HTTP usado para atualizar parcialmente um registo). Cada uma destas rotas recebe o identificador do elemento através do URL e atualiza apenas a coluna visivel, sem alterar mais nenhuma informação. Ambas as rotas estão protegidas pelo middleware apenasAdminMaster, que impede que qualquer utilizador comum consiga aceder a estas ações, mesmo que tente enviar o pedido diretamente para o servidor.

```

musicaAdminApi.js X
projetoFinal > src > backend > API > routes > musicaAdminApi.js > router.patch("/id/mostrar") callback
109
110 // --- PATCH /musicaAdmin/id/esconder -> esconde música ---
111 router.patch("/id/esconder", apenasAdminMaster, async (req, res) => {
112   const { id } = req.params;
113
114   try {
115     // Remove de todas as playlists dos utilizadores antes de esconder
116     await db.query("DELETE FROM PlaylistMusica WHERE IdMusica = ?", [id]);
117     await db.query("UPDATE Musica SET visivel = 0 WHERE IdMusica = ?", [id]);
118
119     console.log("[MUSICA ADMIN] Música ${id} escondida e removida de todas as playlists");
120     res.json({ success: true, message: "Música escondida com sucesso" });
121   } catch (erro) {
122     console.error("Erro ao esconder música:", erro);
123     res.status(500).json({ success: false, message: "Erro interno" });
124   }
125 });
126
127 // --- PATCH /musicaAdmin/id/mostrar -> volta a mostrar música ---
128 router.patch("/id/mostrar", apenasAdminMaster, async (req, res) => {
129   const { id } = req.params;
130
131   try {
132     await db.query("UPDATE Musica SET visivel = 1 WHERE IdMusica = ?", [id]);
133
134     console.log("[MUSICA ADMIN] Música ${id} voltou a ficar visível");
135     res.json({ success: true, message: "Música agora visível" });
136   } catch (erro) {
137     console.error("Erro ao mostrar música:", erro);
138     res.status(500).json({ success: false, message: "Erro interno" });
139   }
140 });
141
142 module.exports = router;

```

Figura 74 - musicaAdminApi.js: 3.8.3

O mesmo padrão repete-se nas rotas de Conteúdo e Produto, apenas mudando o nome da tabela e o campo de identificação (IdConteúdo, IdProduto).

3.9. Gestão de Ficheiros e Uploads (Multer):

Uma parte importante do GoDeeper é a possibilidade de o administrador carregar ficheiros para o servidor - imagens de eventos, imagens de produtos, ficheiros de áudio (MP3) e documentos em PDF. Para tratar estes uploads no *backend*, foi utilizada a biblioteca Multer, que é especificamente desenhada para lidar com formulários que contêm ficheiros no Node.js.

O Multer é configurado individualmente em cada rota que precisa de receber ficheiros, definindo dois aspetos essenciais: a pasta de destino onde o ficheiro será guardado no servidor, e o nome com que o ficheiro será gravado. O nome é gerado automaticamente com base na data e hora atuais, evitando conflitos entre ficheiros com o mesmo nome original.

O módulo de eventos foi o primeiro a utilizar o Multer. Quando o administrador adiciona um novo evento, pode associar uma imagem que será exibida no card do evento. A configuração do Multer para os eventos é a seguinte:

Exemplo – Eventos:

O módulo de eventos foi o primeiro a utilizar o Multer. Quando o administrador adiciona um novo evento, pode associar uma imagem que será exibida no card do evento. A configuração do Multer para os eventos é a seguinte:

```
const multer = require("multer");

// --- Upload de imagem ---
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, "C:\\Users\\gmbis\\Desktop\\PAP\\PAPGabiv.3\\projetoFinal\\src\\fileSystem\\ImagensEventos");
  },
  filename: (req, file, cb) => {
    const nome = Date.now() + path.extname(file.originalname);
    cb(null, nome);
  },
});
const upload = multer({ storage });
```

Figura 75 - eventoApi.js: 3.8.4

A função `Date.now()` devolve o número de milissegundos desde 1970, garantindo que cada nome de ficheiro é único. A extensão original do ficheiro (.jpg, .png, etc.) é preservada através de `path.extname()`. O upload é depois aplicado na rota de adicionar evento:

```
// --- POST /Eventos-GoDeeper/adicionar + só admin_master ---
router.post("/adicionar", apenasAdminMaster, upload.single("imagem"), async (req, res) => {
  const { NomeEvento, LocalEvento, DataEvento, HoraEvento, Descricao } = req.body;
  const CaminhoImagem = req.file ? req.file.filename : null;

  if (!NomeEvento || !DataEvento) {
    return res.status(400).json({ success: false, message: "Nome e data são obrigatórios" });
  }

  try {
    await db.query(`
      INSERT INTO Evento (NomeEvento, LocalEvento, DataEvento, HoraEvento, Descricao, CaminhoImagem)
      VALUES (?, ?, ?, ?, ?, ?)
    `, [NomeEvento, LocalEvento ?? null, DataEvento, HoraEvento || null, Descricao ?? null, CaminhoImagem]);

    console.log(`[EVENTOS] Evento "${NomeEvento}" adicionado`);
    res.json({ success: true, message: "Evento adicionado com sucesso" });
  } catch (erro) {
    console.error("Erro ao adicionar evento:", erro);
    res.status(500).json({ success: false, message: "Erro interno" });
  }
});
```

Figura 76 - eventoApi.js: 3.8.4 p.2

4. PROBLEMAS E SOLUÇÕES ENCONTRADAS

Quais os problemas, não relacionados com falta de conhecimentos básicos, que surgiram durante o Ao longo do desenvolvimento do projeto GoDeeper surgiram vários problemas técnicos que exigiram análise e resolução cuidadosa. De seguida apresentam-se os principais desafios encontrados e as soluções implementadas.

4.1. Gestão de Processos Python Após Reinício do Servidor

Problema: quando o servidor Node.js era reiniciado durante o desenvolvimento, os processos Python eram terminados, mas as sessões dos utilizadores continuavam ativas na base de dados. Ao retomar a utilização, os utilizadores tentavam reproduzir músicas, mas o *player* Python já não estava a correr, resultando em erros de ligação recusada.

```

Erro ao comunicar com dispositivo: FetchError: request to http://localhost:4000/tocarSala failed, reason:
  at ClientRequest.<anonymous> (C:\Users\gmbis\Desktop\PAP\PAPGabiv.3\projetoFinal\node_modules\node-fetch\lib\index.js:1501:11)
  at ClientRequest.emit (node:events:518:28)
  at emitErrorEvent (node:_http_client:104:11)
  at Socket.socketErrorListener (node:_http_client:518:5)
  at Socket.emit (node:events:518:28)
  at emitErrorNT (node:internal/streams/destroy:170:8)
  at emitErrorCloseNT (node:internal/streams/destroy:129:3)
  at process.processTicksAndRejections (node:internal/process/task_queues:90:21) {
  type: 'system',
  errno: 'ECONNREFUSED',
  code: 'ECONNREFUSED'
}
Erro ao comunicar com dispositivo: FetchError: request to http://localhost:4000/pararSala failed, reason:
  at ClientRequest.<anonymous> (C:\Users\gmbis\Desktop\PAP\PAPGabiv.3\projetoFinal\node_modules\node-fetch\lib\index.js:1501:11)
  at ClientRequest.emit (node:events:518:28)
  at emitErrorEvent (node:_http_client:104:11)
  at Socket.socketErrorListener (node:_http_client:518:5)
  at Socket.emit (node:events:518:28)
  at emitErrorNT (node:internal/streams/destroy:170:8)
  at emitErrorCloseNT (node:internal/streams/destroy:129:3)
  at process.processTicksAndRejections (node:internal/process/task_queues:90:21) {
  type: 'system',
  errno: 'ECONNREFUSED',
  code: 'ECONNREFUSED'
}

```

Figura 77 – Problema 1

Solução: foi criado o ficheiro *playerManager.js*, que centraliza a gestão de todos os processos Python. Ao arrancar, o servidor Node.js consulta a base de dados para identificar todos os utilizadores com sessão ativa e inicia automaticamente o processo Python correspondente a cada um deles. Desta forma, os utilizadores podem retomar a utilização sem necessidade de fazer um novo início de sessão.

```
function iniciarPlayerPython(porta) {
  if (playersAtivos[porta]) {
    console.log(`[PLAYER] Já ativo na porta ${porta}`);
    return;
  }

  console.log(`[PLAYER] Caminho do player.py: ${playerPath}`);
  console.log(`[PLAYER] Python path: ${pythonPath}`);
  console.log(`[PLAYER] A iniciar na porta ${porta}...`);

  const processo = spawn(pythonPath, [playerPath, porta], {
    detached: false,
    stdio: "inherit",
  });
}
```

Figura 78 - Resolução1

4.2. Utilização de Await Fora de Funções Assíncronas

Problema: durante a implementação das rotas de controlo do *player* (tocarSala, pararSala, pausarSala), surgiu repetidamente o erro `SyntaxError: await is only valid in async functions`. O problema ocorreu porque o código de consulta à base de dados para obter a porta do dispositivo foi colocado fora da função do router, ao nível do módulo, onde não é válido utilizar `await`.

```
C:\Users\gmbis\Desktop\PAP\PAPGativ.3\projetoFinal\src\backend\API\routes\tocarSala.js:6
const connection = await mysql.createConnection(dbConfig);
                    ^^^^^^

SyntaxError: await is only valid in async functions and the top level bodies of modules
    at wrapSafe (node:internal/modules/cjs/loader:1662:18)
    at Module._compile (node:internal/modules/cjs/loader:1704:20)
    at Object..js (node:internal/modules/cjs/loader:1895:10)
    at Module.load (node:internal/modules/cjs/loader:1465:32)
    at Function.load (node:internal/modules/cjs/loader:1282:12)
    at TracingChannel.traceSync (node:diagnostics_channel:322:14)
    at wrapModuleLoad (node:internal/modules/cjs/loader:235:24)
    at Module.require (node:internal/modules/cjs/loader:1487:12)
    at require (node:internal/modules/helpers:135:16)
    at Object.<anonymous> (C:\Users\gmbis\Desktop\PAP\PAPGativ.3\projetoFinal\src\backend\API\public\server.js:75:25)
Node.js v22.17.1
```

Figura 79 – Problema 2

Solução: a correção consistiu em garantir que toda a lógica de consulta à base de dados, incluindo a obtenção da porta, estava sempre dentro das funções assíncronas do router (router.post('/', async (req, res) => {...})). Este erro foi recorrente ao tentar reutilizar código entre rotas, pelo que foi criada uma estrutura consistente para todas as rotas do projeto.

```
router.post("/", async (req, res)
```

Figura 80 – Resolução 2

4.3. Resolução de Caminhos do Player Python

Problema: o Node.js não conseguia encontrar o ficheiro *player.py* ao tentar iniciar o processo Python. Os caminhos relativos calculados a partir do `__dirname` apresentavam inconsistências dependendo do diretório a partir do qual o servidor era iniciado, resultando num caminho inválido e no erro: `python: can't open file: No such file or directory`.

```
C:\Users\gmbis\Desktop\PAP\PAPGativ.3\projetoFinal\src\backend\API\public>node server.js
Servidor a correr em http://localhost:3000
Player iniciado na porta 4003
python: can't open file 'C:\Users\gmbis\Desktop\PAP\PAPGativ.3\projetoFinal\soundDevices\devicePC5\player.py': [Errno 2] No such file or directory
Player na porta 4003 terminou com código 2
Erro ao comunicar com dispositivo: FetchError: request to http://localhost:4003/tocarSala failed, reason:
  at ClientRequest.<anonymous> (C:\Users\gmbis\Desktop\PAP\PAPGativ.3\projetoFinal\node_modules\node-fetch\lib\index.js:1501:11)
  at ClientRequest.emit (node:events:518:28)
  at emitErrorEvent (node:http_client:104:11)
  at Socket.socketErrorListener (node:http_client:518:5)
  at Socket.emit (node:events:518:28)
  at emitErrorNT (node:internal/streams/destroy:170:8)
  at emitErrorCloseNT (node:internal/streams/destroy:129:3)
  at process.processTicksAndRejections (node:internal/process/task_queues:90:21) {
    type: 'system',
    errno: 'ECONNREFUSED',
    code: 'ECONNREFUSED'
  }
Erro ao comunicar com dispositivo: FetchError: request to http://localhost:4003/pararSala failed, reason:
  at ClientRequest.<anonymous> (C:\Users\gmbis\Desktop\PAP\PAPGativ.3\projetoFinal\node_modules\node-fetch\lib\index.js:1501:11)
  at ClientRequest.emit (node:events:518:28)
  at emitErrorEvent (node:http_client:104:11)
  at Socket.socketErrorListener (node:http_client:518:5)
  at Socket.emit (node:events:518:28)
  at emitErrorNT (node:internal/streams/destroy:170:8)
  at emitErrorCloseNT (node:internal/streams/destroy:129:3)
  at process.processTicksAndRejections (node:internal/process/task_queues:90:21) {
    type: 'system',
    errno: 'ECONNREFUSED',
    code: 'ECONNREFUSED'
  }
Player iniciado na porta 4004
python: can't open file 'C:\Users\gmbis\Desktop\PAP\PAPGativ.3\projetoFinal\soundDevices\devicePC5\player.py': [Errno 2] No such file or directory
Player na porta 4004 terminou com código 2
Erro ao comunicar com dispositivo: FetchError: request to http://localhost:4004/tocarSala failed, reason:
  at ClientRequest.<anonymous> (C:\Users\gmbis\Desktop\PAP\PAPGativ.3\projetoFinal\node_modules\node-fetch\lib\index.js:1501:11)
  at ClientRequest.emit (node:events:518:28)
  at emitErrorEvent (node:http_client:104:11)
  at Socket.socketErrorListener (node:http_client:518:5)
  at Socket.emit (node:events:518:28)
  at emitErrorNT (node:internal/streams/destroy:170:8)
  at emitErrorCloseNT (node:internal/streams/destroy:129:3)
  at process.processTicksAndRejections (node:internal/process/task_queues:90:21) {
    type: 'system',
    errno: 'ECONNREFUSED',
    code: 'ECONNREFUSED'
  }
}
```

Figura 81 – Problema 3

Solução: a solução adotada foi substituir os caminhos relativos por caminhos absolutos no *playerManager.js*: tanto o caminho para o ficheiro *player.py* como o caminho para o executável Python do ambiente virtual (venv). Desta forma, independentemente do diretório de trabalho atual, os caminhos são sempre resolvidos corretamente. A utilização do executável do ambiente virtual garantiu também que todas as bibliotecas Python instaladas (Flask, python-vlc, flask-cors) estavam disponíveis.

```
const pythonPath = "C:\\Users\\gmbis\\Desktop\\PAP\\PAPGativ.3\\projetoFinal\\venv\\Scripts\\python.exe";

const playerPath = path.join(
  __dirname, // src\\backend\\API\\public
  "..",     // src\\backend\\API
  "..",     // src\\backend
  "..",     // src
  "..",     // projetoFinal
  "src",
  "soundDevices",
  "devicePC5",
  "player.py"
);
```

Figura 82 – Resolução 3

4.4. Imagens com Espaços no Nome não carregavam

Problema: as imagens de produtos cujo nome de ficheiro continha espaços não carregavam no browser. O browser interpretava o espaço como um separador de URL inválido, resultando numa imagem quebrada e na apresentação do texto alternativo em vez da fotografia.

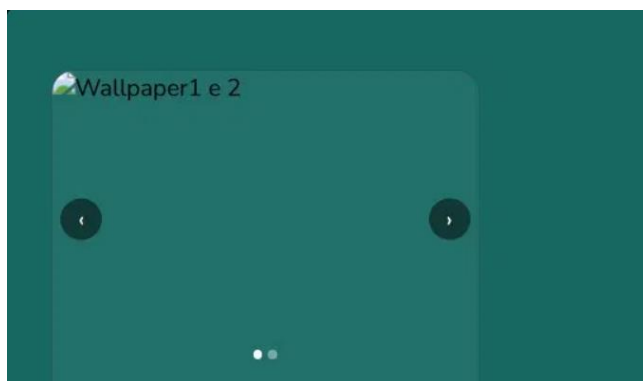


Figura 83 – Problema 4

Solução: a solução foi aplicar a função `encodeURIComponent()` ao nome do ficheiro ao construir os endereços URL das imagens no *frontend*. Esta função converte os caracteres especiais, como os espaços, na sua representação percentual (por exemplo, um espaço torna-se `%20`), produzindo um URL válido que o browser consegue interpretar corretamente.

```
const slides = imagens
  .map(
    (img, i) =>
      ``,
    )
  .join("");
```

Figura 84 – Resolução 4

4.5. Tabela de Sessões Não Existia na Base de Dados

Problema: ao longo do desenvolvimento, diferentes ficheiros do *backend* utilizavam nomes distintos para a base de dados. Alguns ficheiros usavam um nome e outros usavam outro, fazendo com que parte das rotas conseguisse ligar à base de dados e outra parte falhasse com erros de ligação, uma vez que a base de dados referenciada não existia ou não era a correta.

```
Error: Table 'pap_gabi_godeeper_bd.sessions' doesn't exist
at PromisePool.query (C:\Users\gmbis\Desktop\PAP\PAPGabiv.3\projetoFinal\node_modules\express-mysql-session\node_modules\mysql2\promise.js:356:22)
at C:\Users\gmbis\Desktop\PAP\PAPGabiv.3\projetoFinal\node_modules\express-mysql-session\index.js:384:30
at new Promise (<anonymous>)
at MySQLStore.query (C:\Users\gmbis\Desktop\PAP\PAPGabiv.3\projetoFinal\node_modules\express-mysql-session\index.js:381:10)
at C:\Users\gmbis\Desktop\PAP\PAPGabiv.3\projetoFinal\node_modules\express-mysql-session\index.js:170:16
at process.processTicksAndRejections (node:internal/process/task_queues:105:5)
```

Figura 85 – Problema 5

Solução: foi feita uma verificação em todos os ficheiros do *backend* e o nome da base de dados foi uniformizado para `pap_gabi_godeeper_bd` em todo o projeto, em particular nos ficheiros `criarConta.js`, `tocarSala.js`, `pausarSala.js` e `pararSala.js`, garantindo que todos os ficheiros apontavam para a mesma base de dados.

```
const dbConfig = {
  host: "localhost",
  user: "root",
  password: "root",
  database: "pap_gabi_godeeper_bd",
};
```

Figura 86 – Resolução 5

4.6. Dados Undefined nas Inserções na Base de Dados

Problema: ao criar contas de utilizador, surgia o erro `Bind parameters must not contain undefined`. To pass SQL NULL specify JS null. O MySQL não aceita o valor `undefined` do JavaScript; apenas aceita `null` para representar ausência de valor.

```
Bind parameters must not contain undefined. To pass SQL NULL specify JS null
```

Figura 87 – Problema 6

Solução: foi implementado o operador de coalescência nula (`??`) em todos os valores dos parâmetros de inserção na base de dados: `valor ?? null`. Este operador retorna o valor da esquerda se não for `null` nem `undefined`, e retorna `null` caso contrário. Desta forma, todos os campos opcionais não preenchidos são corretamente guardados como `null` na base de dados.


```
// — 1. Criar o utilizador —
const sqlUtilizador = `
  INSERT INTO Utilizador
  (
    nome,
    apelido,
    telemovel,
    email,
    pais,
    NomeUtilizador,
    PasswordUtilizador,
    tipoUtilizador,
    device
  )
  VALUES (?, ?, ?, ?, ?, ?, ?, ?, 'utilizador', 'devicePC5')
`;

const values = [
  nome           ?? null,
  apelido        ?? null,
  telemovel      ?? null,
  email          ?? null,
  pais           ?? null,
  NomeUtilizador ?? null,
  PasswordUtilizador ?? null,
];
```

Figura 88 - Resolução6

5. CONCLUSÃO

O projeto GoDeeper - Mergulhar na Imersão da Palavra de Deus foi concluído com sucesso, cumprindo na íntegra os objetivos definidos na introdução. A plataforma está funcional e inclui todas as funcionalidades planeadas: autenticação de utilizadores com diferentes níveis de acesso, sistema de músicas com *player* de áudio físico independente por utilizador, criação e gestão de playlists personalizadas, módulo de eventos com filtragem por mês, publicação de estudos bíblicos com documentos PDF, loja de produtos com carrossel de imagens e contacto via MBWay (Apenas o número da pessoa a quem deves enviar a mensagem relativamente ao tamanho, à cor e a outras características do produto), edição de dados pessoais e painel de administração.

O projeto cumpre todos os objetivos inicialmente definidos: divulga o grupo de jovens da Igreja Baptista de Caldas da Rainha de forma moderna e acessível, permite a gestão da comunidade pelo líder através de ferramentas de administração integradas na própria plataforma, e oferece um espaço digital de crescimento espiritual para todos os membros.

Funcionalidades Futuras

No futuro, caso a plataforma venha a ter uma utilização efetiva pela comunidade, prevê-se a implementação de diversas melhorias e novas funcionalidades que permitam aumentar a sua utilidade, desempenho e experiência de utilização.

Uma das funcionalidades previstas consiste na implementação de um sistema de notificações por email ou push notifications, permitindo informar os utilizadores sobre novos eventos, conteúdos publicados e produtos disponíveis na plataforma.

Pretende-se igualmente desenvolver um fórum ou espaço de discussão entre os membros, onde seja possível partilhar opiniões, colocar questões e trocar reflexões relacionadas com os estudos bíblicos.

Outra melhoria passa pela criação de uma aplicação móvel nativa para os sistemas Android e iOS, reutilizando a lógica de *backend* já desenvolvida para a plataforma web.

Relativamente à loja de produtos, pretende-se integrar um sistema de pagamentos online, eliminando a necessidade do contacto manual atualmente realizado através de MBWay. Apenas seria necessário o envio de uma mensagem para indicar informações como o tamanho, a cor ou outras características do produto.

Está também prevista a integração de um sistema de transmissão em direto de cultos e eventos, permitindo aos utilizadores assistir aos conteúdos em tempo real através da plataforma.

No que diz respeito aos eventos, pretende-se implementar um sistema de inscrições com confirmação de presença e gestão do número máximo de participantes, permitindo um melhor controlo da lotação.

Para facilitar a administração da plataforma, prevê-se a criação de um painel de estatísticas destinado ao administrador principal, disponibilizando informações como o número de utilizadores ativos, as músicas mais ouvidas, os produtos que despertam maior interesse e os eventos com maior número de visualizações.

Por último, pretende-se disponibilizar suporte multilingue, através da tradução da plataforma para inglês e espanhol, permitindo alcançar utilizadores pertencentes a outras comunidades e alargando o potencial de utilização da aplicação.

Apreciação Final

O desenvolvimento deste projeto representou muito mais do que a conclusão de uma etapa académica, foi um percurso de crescimento real, tanto a nível técnico como pessoal. Cada funcionalidade implementada, cada erro encontrado e cada solução encontrada contribuíram para consolidar conhecimentos em áreas que vão desde o desenvolvimento de APIs REST em Node.js até à integração de serviços externos em Python, passando pela gestão de processos, pelo streaming de áudio e pela arquitetura de sistemas distribuídos.

A construção de uma aplicação full-stack que integra três tecnologias distintas - Node.js, MySQL e Python - não foi uma tarefa simples. Exigiu uma visão global do sistema, uma capacidade de resolução de problemas constante e a humildade de aprender com os erros ao longo do caminho. Desafios como a gestão de processos Python independentes por utilizador, a sincronização de sessões entre o servidor e a base de dados, e a comunicação entre serviços foram, precisamente, os momentos que mais me fizeram crescer como programador.

Olhando para trás, reconheço que cada obstáculo superado foi uma oportunidade de aprendizagem. E olhando para o resultado final, sinto que este projeto representa fielmente o melhor do que fui capaz de construir até hoje e que tudo o que foi construído, foi para a Sua glória.

Avalio a minha prestação de forma muito positiva, pois consegui desenvolver uma aplicação web completa, funcional, com um design cuidado e com funcionalidades de nível profissional, que refletem os valores do grupo GoDeeper e que poderá ser genuinamente utilizada pela comunidade.

Este projeto representou uma oportunidade única de aplicar os conhecimentos adquiridos ao longo do curso num contexto real, com propósito e significado.

A escolha do tema foi intencional e reflete os meus valores: criar algo que sirva não apenas como demonstração de competências técnicas, mas como uma ferramenta genuinamente útil para a comunidade onde me insiro e para a missão de divulgar a Palavra de Deus.

"Porque Dele, por Ele e para Ele são todas as coisas. A Ele seja a glória para sempre. Amém."

Romanos 11:36

REFERÊNCIAS BIBLIOGRÁFICAS

Aceternity UI. (s.d.). *Components*. <https://ui.aceternity.com/components>

Canva. (s.d.). *Templates*. <https://www.canva.com/templates>

Claude. (s.d.). *Claude AI*. Anthropic. <https://claude.ai/new>

Express.js. (s.d.). *Express — Fast, unopinionated, minimalist web framework for Node.js*. <https://expressjs.com/>

iLoveIMG. (s.d.). *Ampliar imagem*. <https://www.iloveimg.com/pt/ampliar-imagem>

MySQL. (s.d.). *MySQL*. Oracle. <https://www.mysql.com/>

Node.js. (s.d.). *Node.js download*. <https://nodejs.org/en/download/current>

Pallets Projects. (s.d.). *Flask*. <https://flask.palletsprojects.com/>

Python Software Foundation. (s.d.). *Python downloads*. <https://www.python.org/downloads/>

Tailwind Labs. (s.d.). *Tailwind CSS*. <https://tailwindcss.com/>

União Batista Portuguesa. (s.d.). *Mapa de igrejas baptistas*. <https://www.baptistasdocentro.pt/mapa>

Uiverse. (s.d.). *Uiverse — UI components*. <https://uiverse.io>

VideoLAN. (s.d.). *VLC media player*. <https://www.videolan.org/>

Visual Studio Code. (s.d.). *Download Visual Studio Code*. Microsoft. <https://code.visualstudio.com/download>

YouTube. (s.d.). *YouTube*. Google. <https://www.youtube.com/>

Claude. (s.d.). *Claude AI*. Anthropic. <https://claude.ai/>

IDENTIFICAÇÃO DO ALUNO

NOME COMPLETO: GABRIEL MARINHO BISPO

N.º DE PROCESSO: 104898

CICLO DE ESTUDOS: ENSINO SECUNDÁRIO

TELEMÓVEL: 928 116 773

E-MAIL: 2025A104898@AERP.PT
